

طبقه‌بندی خودروهای ترافیکی

با استفاده از شبکه‌های عصبی عمیق و یادگیری انتقالی

امیرحسین رنجبر

گزارش پروژه یادگیری عمیق

سخنی پیش از شروع

خوشحالم که فرصت دارید این پژوهش و بخشی از تلاش و ممارست‌هایی را که در مسیر انجام آن داشته‌ام، مشاهده کنید.

پیش از آنکه وارد توضیحات پروژه شویم، لازم می‌دانم به یک نکته اشاره کنم. در تمام مراحل تلاش کرده‌ام کدها و گزارش را با زبانی تا حد امکان سلیس، روان و قابل فهم بنویسم تا مطالعه‌ی آن برای خواننده‌ای مثل شما خسته‌کننده یا آزاردهنده نباشد.

همیشه یکی از اهداف بنده این بوده است که بتوانم دانشی را که در مسیر یادگیری به دست آورده‌ام، در اختیار کسانی قرار دهم که شاید در یادگیری یک موضوع با سختی، پیچیدگی یا سردرگمی مواجه شده‌اند. اگر بتوانم با توضیح ساده‌تر یک مفهوم، حتی اندکی از مسیر یادگیری شما کوتاه‌تر کنم و رسیدن به نتیجه‌ی مطلوب را برایتان آسان‌تر کنم، هدف اصلی این نوشته تا حد زیادی محقق شده است.

به همین دلیل در این گزارش صرفاً به ارائه‌ی نتیجه‌ی نهایی اکتفا نکرده‌ام و سعی کرده‌ام تا حد امکان مسیر رسیدن به آن نتیجه، تصمیم‌هایی که در طول پروژه گرفته شده و دلایلی که پشت هر تصمیم قرار داشته است نیز توضیح داده شود.

البته اگر در بخشی از گزارش توضیحی نامفهوم، بیش از حد پیچیده یا حتی اشتباه وجود داشت، خوشحال می‌شوم آن را با بنده در میان بگذارید. راه ارتباطی و ایمیل بنده در انتهای گزارش قرار داده شده است.

امیدوارم مطالعه‌ی این گزارش برای شما نیز به اندازه‌ی مسیری که برای من در انجام آن لذت‌بخش بود، مفید و لذت‌بخش باشد.

پس بدون هیچ معطلی، شروع کنیم.

چکیده

در این پروژه با مجموعه داده‌ای از تصاویر خودروها مواجه هستیم که از دوربین‌های ترافیکی ثبت شده‌اند. هدف اصلی این است که تصاویر را به یک مدل یادگیری عمیق بدهیم تا مدل بتواند بر اساس ویژگی‌های بصری تصویر، نوع خودرو

را تشخیص داده و آن را در کلاس صحیح قرار دهد.

مجموعه داده‌ی اولیه از سه بخش اصلی تشکیل شده است:

train, unclean و test.

پوشه‌ی train شامل ۸ پوشه است که هر کدام نماینده‌ی یکی از کلاس‌های مسئله بوده و تصاویر مربوط به همان کلاس را در خود جای داده‌اند. پوشه‌ی test نیز ساختاری مشابه دارد و شامل ۸ کلاس است.

اما در پوشه‌ی unclean با ۹ پوشه مواجه هستیم که یکی از آن‌ها مربوط به کلاس «نیسان» است. وجود این کلاس در کنار کلاس «وانت» یک چالش مهم ایجاد می‌کند؛ زیرا از یک طرف برخی از تصاویر موجود در این پوشه می‌توانند با کلاس وانت هم‌پوشانی داشته باشند و از طرف دیگر تصمیم‌گیری درباره‌ی نحوه‌ی استفاده از این تصاویر می‌تواند مستقیماً بر کیفیت داده‌ی آموزشی و در نهایت عملکرد مدل تأثیر بگذارد.

به همین دلیل، برای برخورد با این مجموعه داده سه رویکرد مختلف در نظر گرفته شده است که در ادامه‌ی گزارش، هر یک از آن‌ها را به صورت دقیق بررسی خواهیم کرد.

پیش از ورود به مراحل اصلی، ابتدا لازم است وضعیت داده‌ها را بررسی کنیم و ببینیم در مجموعه‌ی آموزشی، از هر کلاس چه تعداد تصویر در اختیار داریم.

پیش‌پردازش

پیش از شروع هر مرحله‌ی جدی در فرایند آماده‌سازی داده، ابتدا باید مطمئن شویم که فایل‌های موجود در مجموعه داده سالم و قابل خواندن هستند.

بنابراین در اولین مرحله، تمام تصاویر بررسی شدند تا مشخص شود آیا فایل خراب، غیرقابل خواندن یا دارای فرمت نامعتبر در مجموعه وجود دارد یا خیر. نتیجه‌ی این بررسی نشان داد که تصاویر موجود قابل خواندن هستند و فایل خراب یا غیرقابل استفاده‌ای در داده‌ها مشاهده نشد.

پس از اطمینان از سلامت فایل‌ها، مرحله‌ی بعدی شناسایی تصاویر تکراری بود.

این مرحله اهمیت بسیار زیادی دارد؛ زیرا قصد داریم در ادامه از بخشی از تصاویر موجود در پوشه‌ی unclean در کنار تصاویر پوشه‌ی train برای آموزش مدل استفاده کنیم. بنابراین اگر یک تصویر مشابه به صورت هم‌زمان در چند بخش از مجموعه داده وجود داشته باشد، احتمال ایجاد Data Leakage وجود خواهد داشت.

برای مثال، اگر یک تصویر مشخص در مجموعه‌ی آموزش وجود داشته باشد و نمونه‌ی کاملاً یکسانی از همان تصویر در مجموعه‌ی اعتبارسنجی یا آزمون نیز قرار گرفته باشد، مدل در هنگام ارزیابی عملاً با تصویری مواجه می‌شود که قبلاً آن را دیده است. در چنین شرایطی، دقت به‌دست‌آمده می‌تواند غیرواقعی و گمراه‌کننده باشد.

برای جلوگیری از این مشکل، تصاویر موجود در train, test و unclean با استفاده از MD5 Hash بررسی شدند تا نمونه‌های کاملاً تکراری شناسایی شوند.

پس از حذف نمونه‌های تکراری، مرحله‌ی بعدی بررسی ابعاد تصاویر بود.

تصاویر بسیار کوچک می‌توانند اطلاعات بصری کافی برای تشخیص نوع خودرو در اختیار مدل قرار ندهند. به همین دلیل ابتدا توزیع عرض و ارتفاع تصاویر بررسی شد و مقدار ۱۲۰ پیکسل به عنوان یک آستانه‌ی اولیه در نظر گرفته شد.

در این مرحله بررسی شد که آیا تصاویر دارای عرض یا ارتفاع کمتر از ۱۲۰ پیکسل، از نظر انسانی همچنان قابل تشخیص

هستند یا خیر. بررسی نمونه‌ها نشان داد که حتی برخی از تصاویر کوچک‌تر از این مقدار نیز برای تشخیص انسانی قابل استفاده هستند و صرفاً کوچک بودن تصویر نمی‌تواند به تنهایی دلیل کافی برای حذف آن باشد.

بررسی مجموعه داده

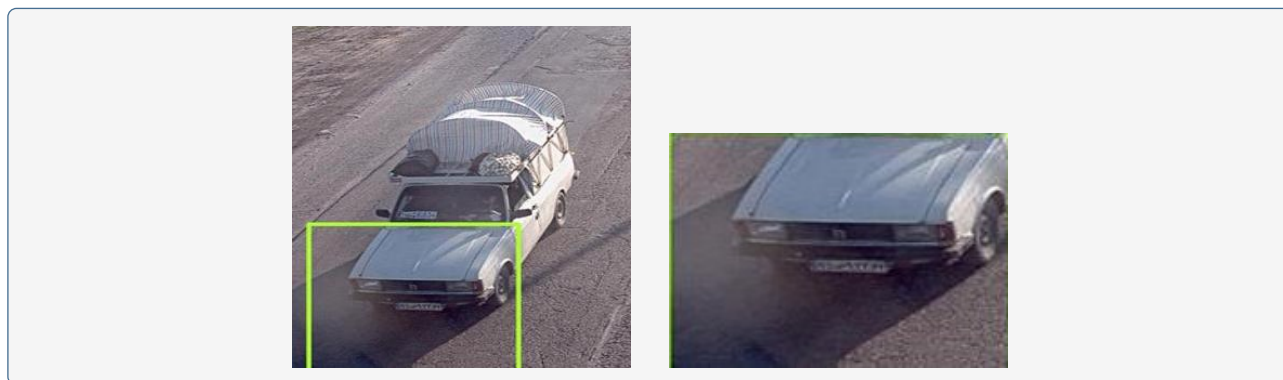
پس از انجام بررسی‌های اولیه، با یک ویژگی غیرمعمول در مجموعه داده مواجه شدیم که توجه ما را به خود جلب کرد. بخش قابل توجهی از تصاویر دارای یک **کادر مستطیلی سبز رنگ** هستند که به منظور مشخص کردن محل خودرو روی تصویر قرار گرفته است.

در نگاه اول، وجود این کادر ممکن است مفید به نظر برسد؛ زیرا به نوعی محل قرارگیری خودرو را مشخص می‌کند. اما مسئله‌ی ما **تشخیص محل خودرو** نیست، بلکه **طبقه‌بندی نوع خودرو** است.

بنابراین این احتمال وجود دارد که مدل به جای یادگیری ویژگی‌های واقعی خودرو، به موقعیت مکانی کادر، شکل آن و یا حتی پیکسل‌های داخل آن وابسته شود.

این موضوع می‌تواند به یک مشکل جدی تبدیل شود؛ زیرا در چنین شرایطی مدل ممکن است به جای اینکه یاد بگیرد چه ویژگی‌هایی یک وانت را از یک خودروی سواری متمایز می‌کنند، صرفاً به الگوهای موجود در ناحیه‌ی داخل کادر وابسته شود.

برای درک بهتر این مسئله، دو تصویر نمونه در ادامه قابل بررسی هستند. اگر تنها بخش داخل کادر را در اختیار مدل قرار دهیم، ممکن است تشخیص نوع خودرو برای آن دشوار شود؛ زیرا برخی از ویژگی‌های مهم خودرو در خارج از این محدوده قرار دارند.



برای مثال، اگر تنها قسمت بریده‌شده‌ی داخل کادر را به فردی نشان دهیم و از او بپرسیم که خودرو وانت است یا پیکان، احتمالاً پاسخ قطعی و یکسانی دریافت نخواهیم کرد.

دلیل این موضوع این است که برای تشخیص نوع خودرو، انسان نیز تنها به یک قسمت محدود از تصویر نگاه نمی‌کند؛ بلکه از مجموعه‌ای از ویژگی‌ها مانند شکل کلی بدنه، کابین، قسمت بار، چراغ‌ها، چرخ‌ها و نسبت بخش‌های مختلف خودرو استفاده می‌کند.

بنابراین اگر مدل ما بیش از حد به پیکسل‌های داخل کادر مستطیلی وابسته شود، ممکن است با یک مشکل مهم مواجه شویم:

مدل به جای یادگیری مفهوم خودرو، به یک نشانه‌ی جانبی در تصویر وابسته شود.

در این حالت، زمانی که تصویر جدیدی وارد مدل شود و ویژگی‌های آن با تصاویر آموزشی متفاوت باشد، احتمال خطای مدل افزایش پیدا می‌کند.

نکته‌ی کلیدی

هدف ما این نیست که مدل صرفاً ناحیه‌ی مشخص‌شده توسط کادر را یاد بگیرد؛ بلکه می‌خواهیم مدل بتواند از مجموعه‌ای از ویژگی‌های بصری خودرو برای تشخیص کلاس آن استفاده کند.

راهکارهای پیشنهادی

با توجه به مسئله‌ی مشاهده‌شده در تصاویر، چند راهکار برای کاهش وابستگی مدل به کادرهای مستطیلی و همچنین افزایش کیفیت داده‌ها در نظر گرفته شد.

۱. حذف کادر و بازسازی ناحیه‌ی حذف‌شده

راه اول این بود که کادر مستطیلی را از تصویر حذف کنیم و قسمت‌هایی را که در اثر حذف کادر از بین می‌روند، بازسازی کنیم.

اما اگر صرفاً کادر را حذف کنیم، بخشی از اطلاعات تصویر نیز از بین خواهد رفت. در نتیجه ممکن است مدل در طول آموزش با تصاویری مواجه شود که دارای قسمت‌های تخریب‌شده هستند.

برای جلوگیری از این مشکل، لازم است نواحی حذف‌شده را بازسازی کنیم تا تصویر نهایی تا حد امکان طبیعی بماند.

این رویکرد اگرچه از نظر مفهومی جذاب است، اما یک هزینه‌ی اضافی نیز به پروژه تحمیل می‌کند؛ زیرا برای بازسازی قسمت‌های حذف‌شده باید از روش‌ها یا مدل‌هایی استفاده کنیم که در زمینه‌ی بازسازی تصاویر عملکرد مناسبی دارند.

در نتیجه، علاوه بر مسئله‌ی اصلی طبقه‌بندی خودرو، یک مسئله‌ی جدید یعنی Image Inpainting نیز وارد پروژه می‌شود که می‌تواند پیچیدگی و هزینه‌ی محاسباتی پروژه را افزایش دهد.

۲. برش تصویر و استفاده‌ی هم‌زمان از تصویر اصلی

راه دوم این بود که تصاویر را بر اساس کادر موجود برش دهیم و نسخه‌ی برش‌خورده‌ی خودرو را نیز در اختیار مدل قرار دهیم؛ اما در کنار آن، تصویر اصلی که کادر مستطیلی را در خود دارد نیز حفظ شود.

ایده‌ی اصلی این رویکرد این است که مدل تنها به یک نوع نمایش از تصویر وابسته نباشد.

از یک طرف، تصویر اصلی اطلاعات کلی صحنه و ویژگی‌های اطراف خودرو را در اختیار مدل قرار می‌دهد و از طرف دیگر، تصویر برش‌خورده تمرکز بیشتری روی خود خودرو ایجاد می‌کند.

به این ترتیب امیدواریم مدل به جای وابستگی شدید به کادر مستطیلی، بتواند ویژگی‌های واقعی و قابل تعمیم خودرو را یاد بگیرد.

۳. پاک‌سازی داده‌ها و حذف نمونه‌های نامناسب

راهکار دیگر، بررسی دقیق تصاویر و حذف نمونه‌هایی بود که می‌توانند باعث ایجاد نویز در فرآیند آموزش شوند. در این مرحله، داده‌های پرت و نمونه‌های نامناسب شناسایی و حذف شدند. همچنین تصاویر مربوط به خودروهای نیسان که در پوشه‌ی vanet قرار داشتند، بررسی شدند؛ زیرا وجود این تصاویر می‌توانست باعث شود مدل ویژگی‌های نامرتب با کلاس اصلی را یاد بگیرد. هدف از این مرحله، ایجاد مجموعه‌داده‌ای با کیفیت بالاتر و کاهش نمونه‌هایی بود که می‌توانستند باعث ایجاد خطا در فرآیند یادگیری شوند.

۴. مقایسه‌ی چند نسخه از مجموعه‌داده

در نهایت، به جای انتخاب یک نسخه از داده‌ها بدون مقایسه‌ی تجربی، چند نسخه‌ی مختلف از مجموعه‌داده آماده شد و عملکرد مدل روی آن‌ها مورد بررسی قرار گرفت. به این ترتیب می‌توان تأثیر تعداد تصاویر، برش تصاویر، پاک‌سازی داده‌ها و حذف نمونه‌های نامناسب را بر عملکرد مدل بررسی کرد. این روش امکان می‌دهد تصمیم نهایی درباره‌ی مجموعه‌داده بر اساس نتایج آزمایش‌ها انجام شود و نه صرفاً بر اساس فرض اولیه.

معرفی نسخه‌های مختلف مجموعه‌داده

پس از بررسی مشکلات موجود در داده‌ها و بررسی راهکارهای مختلف، سه نسخه‌ی متفاوت از مجموعه‌داده برای انجام آزمایش‌ها آماده شد. هدف از ایجاد این نسخه‌ها، بررسی تأثیر نحوه‌ی آماده‌سازی داده‌ها، برش تصاویر و پاک‌سازی نمونه‌ها بر عملکرد مدل بود.

دیتاست اول: ادغام Train و Unclean

در نسخه‌ی اول، داده‌های موجود در دو بخش Train و Unclean با یکدیگر ادغام شدند. هدف اصلی این مرحله، ایجاد یک مجموعه‌داده‌ی بزرگ‌تر و استفاده از تعداد بیشتری نمونه برای آموزش مدل بود. تعداد تصاویر هر کلاس در این نسخه به صورت زیر است:

Images	Class
۳۶۲	ambulance
۴۷۰	autobus
۴۹۶	kamyun
۴۵۵	kamyunet
۴۱۱	minibus
۵۰۰	savari
۴۹۳	taxi
۵۰۰	vanet

دیتاست دوم: افزودن و ایجاد تصاویر برش‌خورده

در نسخه‌ی دوم، علاوه بر تصاویر موجود در مجموعه‌داده، تعدادی تصویر برش‌خورده نیز به مجموعه‌داده اضافه شدند. همچنین بخشی از تصاویر موجود بر اساس ناحیه‌ی خودرو برش داده شدند تا نمایش متمرکزتری از خودرو در اختیار مدل قرار گیرد.

هدف از این کار، کاهش تأثیر بخش‌های غیرضروری تصویر و فراهم کردن نمونه‌هایی بود که تمرکز بیشتری بر خود خودرو دارند. بنابراین در این نسخه، مدل علاوه بر تصاویر اصلی، با تصاویر برش‌خورده نیز مواجه می‌شود و می‌تواند از هر دو نوع نمایش برای یادگیری ویژگی‌های خودرو استفاده کند.

تصاویر برش‌خورده دارای ساختار مستطیلی مشخصی هستند و ابعاد آن‌ها پس از برش یکسان‌سازی شدند. در نتیجه، تعداد تصاویر این نسخه نسبت به دیتاست اول افزایش پیدا کرده است.

تعداد تصاویر هر کلاس در این نسخه به صورت زیر است:

Images	Class
۴۴۶	ambulance
۵۶۹	autobus
۵۹۶	kamyun
۵۵۱	kamyunet
۵۰۶	minibus
۵۹۸	savari
۵۸۹	taxi
۵۹۸	vanet

در مجموع، این نسخه با ترکیب تصاویر اصلی و تصاویر برش‌خورده، تعداد بیشتری نمونه نسبت به دیتاست اول در اختیار مدل قرار می‌دهد و امکان بررسی تأثیر استفاده از تصاویر برش‌خورده را فراهم می‌کند.

دیتاست سوم: ادغام داده‌ها و پاک‌سازی نهایی

در نسخه‌ی سوم نیز داده‌های موجود در بخش‌های Unclean و Train با یکدیگر ادغام شدند؛ با این تفاوت که پس از ادغام، فرآیند پاک‌سازی دقیق‌تری روی داده‌ها انجام گرفت.

در این مرحله، داده‌های پرت و نمونه‌های نامناسب شناسایی و حذف شدند. همچنین تصاویر مربوط به خودروهای نیسان که در پوشه‌ی vanet قرار داشتند، از این پوشه حذف شدند.

هدف از این کار جلوگیری از ورود نمونه‌هایی بود که با مفهوم اصلی کلاس vanet سازگاری کافی نداشتند و می‌توانستند باعث ایجاد ابهام در فرآیند یادگیری مدل شوند.

در نتیجه، دیتاست سوم از نظر تعداد نمونه‌ها کوچک‌تر از دیتاست دوم است، اما داده‌های آن پس از فرآیند پاک‌سازی، کیفیت و سازگاری بیشتری با مسئله‌ی طبقه‌بندی مورد نظر دارند.

تعداد تصاویر باقی‌مانده در هر کلاس در این نسخه به صورت زیر است:

Images	Class
۴۴۵	ambulance
۵۶۹	autobus
۵۹۶	kamyun
۵۵۱	kamyunet
۵۰۵	minibus
۵۹۸	savari
۵۸۷	taxi
۱۸۰	vanet

مقایسه‌ی سه نسخه‌ی مجموعه‌داده

برای مقایسه‌ی بهتر، تعداد کل تصاویر موجود در هر نسخه محاسبه شد:

Images Total	Dataset
۳۶۸۷	Dataset 1
۴۴۵۳	Dataset 2
۴۰۵۵	Dataset 3

دیتاست دوم بیشترین تعداد تصویر را در اختیار دارد، در حالی که دیتاست سوم پس از حذف داده‌های پرت و نمونه‌های نامناسب، مجموعه‌ای کوچک‌تر اما پاک‌سازی‌شده‌تر ایجاد کرده است.

این تفاوت میان سه نسخه‌ی داده اهمیت زیادی دارد؛ زیرا عملکرد مدل تنها به معماری شبکه وابسته نیست و کیفیت، تعداد و نحوه‌ی آماده‌سازی داده‌ها نیز می‌تواند تأثیر مستقیمی بر نتیجه‌ی نهایی داشته باشد.

جمع‌بندی این مرحله

در این بخش ابتدا ویژگی‌های مجموعه داده و وجود کادرهای مستطیلی سبز رنگ بررسی شد. سپس راهکارهای مختلفی برای کاهش وابستگی مدل به این کادرها و همچنین بهبود کیفیت داده‌ها مطرح شد. در ادامه، سه نسخه متفاوت از مجموعه داده ایجاد شدند. در دیتاست اول، داده‌های Train و Unclean ادغام شدند. در دیتاست دوم، تصاویر بر اساس ناحیه‌ی خودرو برش داده شدند. در دیتاست سوم نیز داده‌های Train و Unclean ادغام شده و پس از آن داده‌های پرت و نمونه‌های نامناسب حذف شدند. همچنین تصاویر مربوط به خودروهای نیسان از پوشه‌ی vanet حذف شدند. این سه نسخه امکان مقایسه‌ی اثر افزایش داده، برش تصاویر و پاک‌سازی داده‌ها را بر عملکرد مدل فراهم می‌کنند.

معماری شبکه عصبی سفارشی

پس از آماده‌سازی و بررسی مجموعه داده، در مرحله‌ی بعد لازم است معماری شبکه‌ای را طراحی کنیم که بتواند ویژگی‌های بصری خودروها را از تصاویر استخراج کرده و آن‌ها را در یکی از ۸ کلاس موجود قرار دهد.

در این پروژه از یک معماری شبکه عصبی کانولوشنی سفارشی با نام CustomCNN استفاده شده است. این شبکه از ابتدا برای مسئله‌ی طبقه‌بندی تصاویر خودرو طراحی شده و برخلاف مدل‌های ازپیش‌آموزش‌دیده مانند ResNet18، وزن‌های آن از ابتدا و بر اساس داده‌های همین پروژه یاد گرفته می‌شوند.

ورودی شبکه تصاویر رنگی با اندازه‌ی 150×150 پیکسل است.

معماری شبکه از دو بخش اصلی تشکیل شده است:

۱. بخش استخراج ویژگی (Feature Extractor)

۲. بخش طبقه‌بندی (Classifier)

بخش استخراج ویژگی

بخش استخراج ویژگی از چهار مرحله‌ی اصلی تشکیل شده است. در هر مرحله دو لایه‌ی کانولوشنی قرار گرفته و پس از هر لایه‌ی کانولوشنی از Batch Normalization و تابع فعال‌ساز ReLU استفاده شده است.

در انتهای سه مرحله‌ی ابتدایی نیز از Max Pooling با اندازه‌ی 2×2 استفاده شده است.

ساختار کلی این بخش به صورت زیر است:

Conv(۳ → ۳۲) → BN → ReLU → Conv(۳۲ → ۳۲)
→ BN → ReLU → MaxPool

Conv(۳۲ → ۶۴) → BN → ReLU → Conv(۶۴ → ۶۴)
→ BN → ReLU → MaxPool

Conv(۶۴ → ۱۲۸) → BN → ReLU → Conv(۱۲۸ → ۱۲۸)
→ BN → ReLU → MaxPool

Conv(۱۲۸ → ۲۵۶) → BN → ReLU → Conv(۲۵۶ → ۲۵۶)
→ BN → ReLU

در این معماری، تعداد کانال‌های ویژگی به صورت تدریجی از ۳۲ به ۶۴، سپس ۱۲۸ و در نهایت ۲۵۶ افزایش پیدا می‌کند. این افزایش باعث می‌شود شبکه بتواند در لایه‌های عمیق‌تر ویژگی‌های پیچیده‌تر و سطح بالاتری از تصاویر را استخراج کند.

از طرف دیگر، استفاده از Max Pooling باعث کاهش ابعاد مکانی تصویر می‌شود. بنابراین با افزایش عمق شبکه، اندازه‌ی فضایی ویژگی‌ها کاهش پیدا کرده و تعداد کانال‌های ویژگی افزایش می‌یابد.

تغییر ابعاد ویژگی‌ها

از آنجا که اندازه‌ی تصویر ورودی ۱۵۰×۱۵۰ است، ابعاد ویژگی‌ها پس از هر عملیات Max Pooling به صورت زیر تغییر می‌کند:

مرحله	تعداد کانال	ابعاد ویژگی
ورودی	۳	۱۵۰×۱۵۰
بعد از Pooling اول	۳۲	۷۵ × ۷۵
بعد از Pooling دوم	۶۴	۳۷ × ۳۷
بعد از Pooling سوم	۱۲۸	۱۸ × ۱۸
خروجی Extractor Feature	۲۵۶	۱۸ × ۱۸

در نتیجه، خروجی بخش استخراج ویژگی دارای ابعاد

$$۲۵۶ \times ۱۸ \times ۱۸$$

خواهد بود.

سپس این ویژگی‌ها توسط لایه Flatten به یک بردار یک‌بعدی با اندازه‌ی

$$256 \times 18 \times 18 = 82944$$

تبدیل می‌شوند.

بخش طبقه‌بندی

پس از استخراج ویژگی‌ها، بردار حاصل وارد بخش طبقه‌بندی می‌شود.

در اولین لایه‌ی تمام‌متصل، تعداد ویژگی‌ها از ۸۲۹۴۴ به ۲۵۶ کاهش پیدا می‌کند. پس از آن تابع فعال‌ساز ReLU و سپس Dropout با نرخ ۵۰٪ قرار گرفته است.

استفاده از Dropout باعث می‌شود در هنگام آموزش، بخشی از نورون‌ها به صورت تصادفی غیرفعال شوند. هدف از این کار کاهش احتمال Overfitting و افزایش توانایی تعمیم مدل است.

در نهایت، یک لایه‌ی تمام‌متصل دیگر با ۸ خروجی قرار گرفته است؛ زیرا مسئله‌ی ما شامل ۸ کلاس مختلف خودرو است.

ساختار این بخش به صورت زیر است:

$$\text{Flatten} \rightarrow \text{Linear}(256, 82944) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(50\%) \rightarrow \text{Linear}(8, 256)$$

خروجی نهایی شبکه شامل ۸ مقدار است که هر مقدار مربوط به یکی از کلاس‌های خودرو است.

تعداد پارامترهای شبکه

یکی از نکات مهم در بررسی یک معماری شبکه عصبی، تعداد پارامترهای آن است.

در این شبکه، مجموع پارامترهای قابل آموزش برابر با

$$22,410,152$$

پارامتر است.

بخش بسیار بزرگی از این پارامترها مربوط به اولین لایه‌ی تمام‌متصل بخش طبقه‌بندی است.

این لایه دارای

$$21,233,920$$

پارامتر است.

برای مقایسه، تعداد پارامترهای بخش کانولوشنی بسیار کمتر از این مقدار است. این موضوع نشان می‌دهد که اگرچه شبکه از چندین لایه کانولوشنی تشکیل شده است، بخش قابل توجهی از حجم پارامترهای مدل مربوط به اتصال ویژگی‌های استخراج شده به لایه‌ی تمام‌متصل است.

بخش	تعداد پارامترها
لایه‌های کانولوشنی و Normalization Batch	۱,۱۷۳,۹۲۰
لایه‌ی Linear اول	۲۱,۲۳۳,۹۲۰
لایه‌ی Linear نهایی	۲,۰۵۶
مجموع	۲۲,۴۱۰,۱۵۲

بنابراین معماری نهایی ما را می‌توان به صورت خلاصه به شکل زیر معرفی کرد:

نام مدل: CustomCNN
 نوع مدل: شبکه عصبی کانولوشنی سفارشی
 اندازه ورودی: $3 \times 150 \times 150$
 تعداد کلاس‌ها: ۸
 تعداد کل پارامترها: ۲۲,۴۱۰,۱۵۲
 پارامترهای قابل آموزش: ۲۲,۴۱۰,۱۵۲
 تابع فعال‌ساز: ReLU
 Regularization: Dropout با نرخ ۵.۰

آموزش و تحلیل عملکرد شبکه عصبی سفارشی

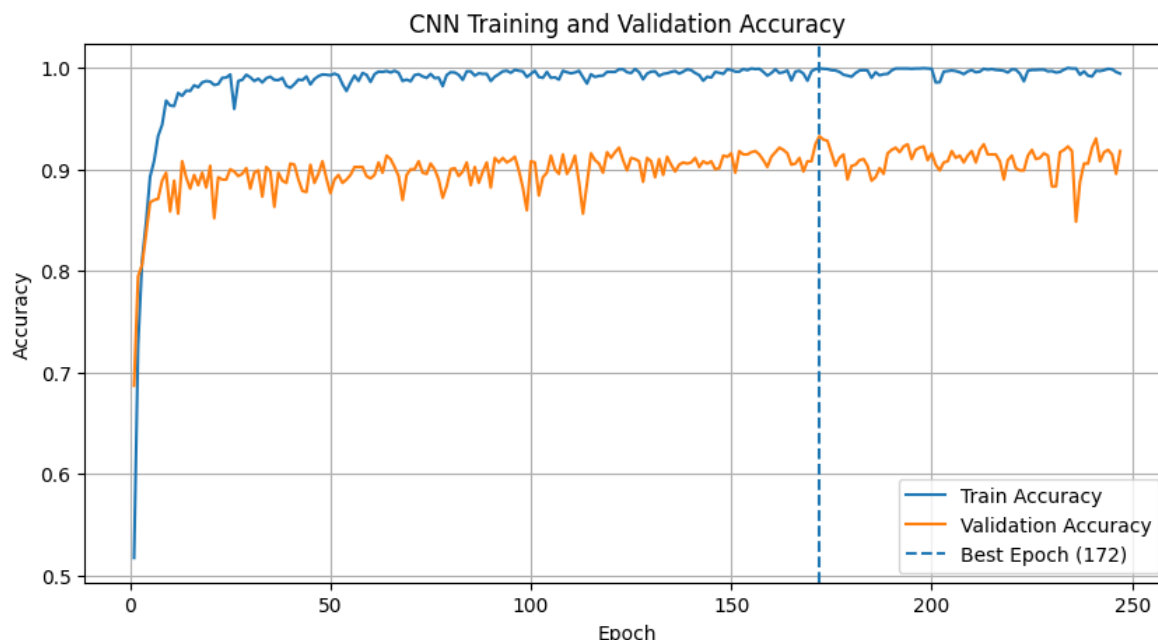
پس از طراحی معماری CustomCNN، لازم بود عملکرد آن در فرایند آموزش و اعتبارسنجی بررسی شود. هدف از این آزمایش، مشاهده‌ی روند یادگیری شبکه، بررسی میزان تعمیم مدل و شناسایی نشانه‌های احتمالی Overfitting بود.

در این آزمایش، همان معماری CustomCNN معرفی شده در بخش قبل مورد استفاده قرار گرفت. این شبکه شامل چهار بلوک کانولوشنی، لایه‌های Batch Normalization و ReLU و بخش طبقه‌بندی تمام‌متصل است. ورودی شبکه تصاویر رنگی با اندازه‌ی 150×150 پیکسل و خروجی آن شامل ۸ کلاس خودرو است. در این آزمایش، مدل با روش Adam و نرخ یادگیری 10^{-4} آموزش داده شد.

برای آموزش مدل حداکثر ۵۰۰ epoch در نظر گرفته شد. با این حال، به منظور جلوگیری از ادامه‌ی آموزش پس از متوقف شدن بهبود عملکرد مدل، از Early Stopping با مقدار $\text{patience} = 75$ استفاده شد. بنابراین اگر عملکرد مدل روی مجموعه‌ی اعتبارسنجی برای ۷۵ epoch متوالی بهبود پیدا نمی‌کرد، آموزش متوقف می‌شد. این روش باعث می‌شود در صورت رسیدن مدل به ناحیه‌ای که یادگیری جدید سود قابل توجهی ایجاد نمی‌کند، از آموزش غیرضروری جلوگیری شود.

روند دقت آموزش و اعتبارسنجی

شکل زیر روند تغییرات Train Accuracy و Validation Accuracy را در طول آموزش نشان می‌دهد. خط چین عمودی محل بهترین عملکرد مدل روی مجموعه‌ی اعتبارسنجی را مشخص می‌کند.



شکل ۱: روند دقت آموزش و اعتبارسنجی شبکه CustomCNN. خط عمودی محل بهترین Validation Accuracy در epoch شماره ۱۷۲ را نشان می‌دهد.

همان‌طور که در شکل ۱ مشاهده می‌شود، شبکه در epoch‌های ابتدایی با سرعت زیادی یاد می‌گیرد. در شروع آموزش، دقت آموزش و اعتبارسنجی از مقادیر پایین آغاز شده و در چند epoch نخست افزایش قابل توجهی دارند. این رفتار نشان می‌دهد که شبکه در ابتدای آموزش به سرعت ویژگی‌های اولیه و قابل استفاده‌ی تصاویر را یاد می‌گیرد.

پس از این افزایش اولیه، Train Accuracy به تدریج به مقادیر بسیار بالا نزدیک می‌شود و در بخش بزرگی از آموزش در محدوده‌ی حدود ۹۹ تا ۱۰۰ درصد قرار می‌گیرد. در مقابل، Validation Accuracy در محدوده‌ی تقریبی ۸۹ تا ۹۳ درصد نوسان می‌کند. بنابراین، اگرچه مدل توانسته است داده‌های آموزشی را تقریباً به طور کامل یاد بگیرد، عملکرد آن روی داده‌های دیده‌نشده با فاصله‌ای قابل توجه پایین‌تر باقی مانده است.

این فاصله میان دقت آموزش و اعتبارسنجی یکی از نشانه‌های مهم Overfitting است. به بیان ساده، مدل در یادگیری نمونه‌های آموزشی بسیار موفق عمل کرده است، اما بخشی از الگوهایی که یاد گرفته است به اندازه‌ی کافی قابل تعمیم به تصاویر جدید نیستند. با توجه به معماری مورد استفاده، این رفتار قابل انتظار نیز هست؛ زیرا شبکه‌ی CustomCNN حدود ۴.۲۲ میلیون پارامتر قابل آموزش دارد و ظرفیت نسبتاً بالایی برای یادگیری جزئیات مجموعه داده دارد.

بهترین epoch و Stopping Early

مطابق شکل، بهترین عملکرد اعتبارسنجی در **epoch شماره ۱۷۲** به دست آمده است. در این نقطه، Validation Accuracy به حدود ۹۳ درصد رسیده و بالاترین مقدار ثبت‌شده در طول این اجرای آموزشی را تشکیل می‌دهد. بنابراین وزن‌های مربوط به این epoch به عنوان بهترین وضعیت مدل در نظر گرفته می‌شوند.

پس از epoch شماره ۱۷۲، اگرچه دقت آموزش همچنان در محدوده‌ی بسیار بالا باقی می‌ماند، دقت اعتبارسنجی روند افزایشی پایدار و معناداری نشان نمی‌دهد و بیشتر حول محدوده‌ی ۹۰ تا ۹۲ درصد نوسان می‌کند. به همین دلیل، پس

از گذشت epoch ۷۵ بدون بهبود در بهترین دقت اعتبارسنجی، شرط Early Stopping فعال شده و آموزش تقریباً در epoch شماره ۲۴۷ متوقف شده است.

این موضوع اهمیت Early Stopping را نشان می‌دهد. اگرچه حداکثر epoch ۵۰۰ برای آموزش تعیین شده بود، مدل عملاً نیازی به اجرای تمام epoch ۵۰۰ نداشت. در واقع، ادامه‌ی آموزش پس از بهترین epoch عمدتاً باعث افزایش دقت آموزش می‌شد، در حالی که بهبود پایداری در مجموعه‌ی اعتبارسنجی مشاهده نمی‌شد. بنابراین توقف زودهنگام از انجام آموزش‌های اضافی جلوگیری کرده و بهترین وضعیت مدل را نیز حفظ کرده است.

جمع‌بندی رفتار مدل

بر اساس روند مشاهده‌شده، می‌توان رفتار شبکه را در سه مرحله خلاصه کرد:

۱. **یادگیری سریع در ابتدای آموزش:** دقت آموزش و اعتبارسنجی در epoch ابتدایی با سرعت زیادی افزایش پیدا می‌کنند که نشان‌دهنده‌ی یادگیری سریع الگوهای اصلی تصاویر است.

۲. **رسیدن به عملکرد بالا روی داده‌های آموزش:** دقت آموزش پس از مدت کوتاهی به حدود ۹۹ تا ۱۰۰ درصد نزدیک می‌شود. این موضوع نشان می‌دهد که ظرفیت مدل برای یادگیری داده‌های آموزشی بالا است.

۳. **ایجاد فاصله میان آموزش و اعتبارسنجی:** دقت اعتبارسنجی در محدوده‌ی پایین‌تری باقی می‌ماند و نوسان آن پس از epoch میانی افزایش پیدا می‌کند. این فاصله میان دو منحنی نشان‌دهنده‌ی وجود Overfitting و محدودیت در تعمیم مدل است.

در مجموع، این آزمایش نشان می‌دهد که CustomCNN توانایی یادگیری بسیار خوبی روی مجموعه‌ی آموزش دارد، اما برای افزایش توانایی تعمیم آن می‌توان در آزمایش‌های بعدی از روش‌هایی مانند Data Augmentation، Weight Decay، تنظیم نرخ یادگیری و روش‌های منظم‌سازی مناسب‌تر استفاده کرد. همچنین مقایسه‌ی این مدل با معماری‌های از پیش‌آموزش‌دیده مانند ResNet18 می‌تواند نشان دهد که آیا استفاده از ویژگی‌های آموخته‌شده از مجموعه‌داده‌های بزرگ‌تر می‌تواند عملکرد بهتری روی این مسئله ایجاد کند یا خیر.

جمع‌بندی آزمایش: حداکثر تعداد epoch برابر با ۵۰۰ در نظر گرفته شد، اما با استفاده از Early Stopping با $\text{patience}=75$ ، آموزش پس از عدم بهبود طولانی‌مدت متوقف شد. بهترین عملکرد اعتبارسنجی در epoch شماره ۱۷۲ به دست آمد و وزن‌های همین epoch به عنوان بهترین وضعیت مدل ذخیره شدند. فاصله‌ی قابل توجه میان دقت آموزش و اعتبارسنجی نیز نشان می‌دهد که در این آزمایش، مدل با وجود یادگیری بسیار خوب داده‌های آموزش، با مسئله‌ی Overfitting مواجه بوده است.

آزمایش متعادل‌سازی Batch ها با استفاده از Sampling Batch Balanced

در آزمایش قبلی، مدل CustomCNN با استفاده از داده‌های آموزشی موجود آموزش داده شد. یکی از نکاتی که در مسائل طبقه‌بندی چندکلاسه اهمیت زیادی دارد، نحوه‌ی توزیع نمونه‌ها در میان کلاس‌های مختلف است.

اگر تعداد تصاویر مربوط به کلاس‌های مختلف یکسان نباشد، ممکن است مدل در هنگام آموزش، نمونه‌های مربوط به

کلاس‌های پرتعداد را بسیار بیشتر از کلاس‌های کم‌تعداد مشاهده کند. در چنین شرایطی، مدل فرصت بیشتری برای یادگیری الگوهای کلاس‌های پرتعداد خواهد داشت و ممکن است در تشخیص کلاس‌های کم‌تعداد عملکرد ضعیف‌تری پیدا کند.

به همین دلیل، در آزمایش بعدی از روشی به نام **Balanced Batch Sampling** استفاده می‌کنیم.

Batch Balanced چیست؟

برای درک مفهوم **Balanced Batch** ابتدا باید بدانیم **Batch** چیست.

در فرایند آموزش شبکه عصبی، معمولاً تمام تصاویر مجموعه‌ی آموزشی به صورت هم‌زمان وارد شبکه نمی‌شوند. داده‌ها به گروه‌های کوچک‌تری تقسیم می‌شوند که هر گروه یک **Batch** نام دارد.

برای مثال، اگر اندازه‌ی **Batch** برابر ۳۲ باشد، در هر مرحله ۳۲ تصویر به شبکه داده می‌شود و مدل بر اساس همین ۳۲ تصویر وزن‌های خود را به‌روزرسانی می‌کند.

در حالت عادی، انتخاب این ۳۲ تصویر می‌تواند باعث شود تعداد نمونه‌های مربوط به کلاس‌های مختلف در هر **Batch** یکسان نباشد. برای مثال ممکن است یک **Batch** شامل ۱۵ تصویر از یک کلاس، ۸ تصویر از کلاس دیگر و تنها ۱ یا ۲ تصویر از یک کلاس کم‌تعداد باشد.

در این حالت، سهم کلاس‌های مختلف از فرآیند یادگیری برابر نیست.

هدف **Balanced Batch Sampling** این است که این وضعیت را کنترل کند و **Batch**‌هایی بسازد که نمونه‌های کلاس‌های مختلف در آن‌ها تا حد امکان به صورت متعادل حضور داشته باشند.

در مسئله‌ی ما که شامل ۸ کلاس خودرو است، می‌توان یک **Batch** با اندازه‌ی ۳۲ را به گونه‌ای تشکیل داد که از هر کلاس تقریباً ۴ تصویر در آن وجود داشته باشد:

$$\text{تصویر ۳۲} = \text{تصویر از هر کلاس ۴} \times \text{کلاس ۸}$$

در نتیجه، به جای اینکه ترکیب هر **Batch** به صورت تصادفی و وابسته به تعداد نمونه‌های هر کلاس باشد، تلاش می‌شود هر کلاس سهم مشخص و تقریباً برابری از **Batch** داشته باشد.

چرا از Batch Balanced استفاده می‌کنیم؟

دلیل اصلی انجام این آزمایش، بررسی تأثیر متعادل‌سازی فرآیند آموزش بر عملکرد مدل است.

فرض کنیم تعداد تصاویر بعضی از کلاس‌ها بیشتر از کلاس‌های دیگر باشد. در روش معمول، کلاس‌های پرتعداد احتمال بیشتری دارند که در **Batch**‌های مختلف ظاهر شوند. بنابراین مدل در طول آموزش، نمونه‌های بیشتری از این کلاس‌ها مشاهده می‌کند.

این مسئله می‌تواند باعث ایجاد نوعی **Class Bias** شود؛ یعنی مدل به مرور در تشخیص کلاس‌های پرتعداد عملکرد بهتری پیدا کند، در حالی که کلاس‌های کم‌تعداد سهم کمتری از فرآیند یادگیری داشته باشند.

با استفاده از **Balanced Batch Sampling** می‌خواهیم این اثر را کاهش دهیم.

در این روش، حتی اگر تعداد واقعی تصاویر کلاس‌ها متفاوت باشد، تلاش می‌کنیم در سطح **Batch** سهم کلاس‌ها متعادل باشد. بنابراین مدل در هر مرحله‌ی آموزش فرصت بیشتری برای مشاهده و یادگیری نمونه‌های مربوط به تمام

کلاس‌ها خواهد داشت.

چه اتفاقی در فرآیند آموزش انتظار داریم؟

انتظار نداریم استفاده از Balanced Batch الزاماً باعث افزایش مستقیم دقت کلی مدل شود. هدف اصلی این آزمایش، متعادل‌تر کردن فرآیند یادگیری میان کلاس‌ها است.

بنابراین چند اتفاق ممکن است مشاهده شود.

اولین مورد این است که عملکرد مدل روی کلاس‌هایی که در حالت عادی نمونه‌های کمتری دارند، بهبود پیدا کند. دلیل آن این است که این کلاس‌ها در طول آموزش بیشتر و منظم‌تر در Batch حضور خواهند داشت.

دومین مورد می‌تواند کاهش اختلاف عملکرد میان کلاس‌های مختلف باشد. برای مثال، ممکن است در حالت عادی مدل روی یک کلاس دقت بسیار بالایی داشته باشد اما روی کلاس دیگری Recall پایینی داشته باشد. با متعادل شدن Batch انتظار داریم این اختلاف تا حدی کاهش پیدا کند.

از طرف دیگر، ممکن است Accuracy کلی مدل حتی کاهش پیدا کند. این موضوع لزوماً به معنی بدتر شدن مدل نیست. زیرا ممکن است مدل با استفاده از Batch Balanced از تمرکز بیش از حد روی کلاس‌های پرتعداد فاصله گرفته و عملکرد متعادل‌تری روی تمام کلاس‌ها ایجاد کند.

به همین دلیل در این آزمایش تنها به Accuracy کلی توجه نمی‌کنیم و معیارهایی مانند Macro Precision, Macro Recall و Macro F1 را نیز بررسی خواهیم کرد.

نکته‌ی مهم در تفسیر نتایج

یکی از مهم‌ترین نکات این آزمایش این است که افزایش دقت کلی به تنهایی معیار مناسبی برای قضاوت درباره‌ی موفقیت Balanced Batch نیست.

برای مثال، فرض کنیم مدل قبل از متعادل‌سازی روی کلاس‌های پرتعداد عملکرد بسیار خوبی داشته باشد، اما روی چند کلاس کم‌تعداد عملکرد ضعیفی نشان دهد. در چنین شرایطی ممکن است دقت کلی همچنان مقدار بالایی داشته باشد.

اما اگر پس از استفاده از Balanced Batch، عملکرد کلاس‌های ضعیف‌تر بهتر شود و در نتیجه Macro F1 افزایش پیدا کند، می‌توان نتیجه گرفت که مدل توانسته است عملکرد متعادل‌تری میان کلاس‌ها ایجاد کند؛ حتی اگر Accuracy کلی تغییر زیادی نکرده باشد یا حتی کمی کاهش پیدا کند.

بنابراین در این آزمایش، علاوه بر عملکرد کلی مدل، به عملکرد هر یک از ۸ کلاس نیز توجه خواهیم کرد.

انتظار کلی از آزمایش

به صورت خلاصه، انتظار ما از این آزمایش این است که:

۱. سهم کلاس‌های مختلف در Batch آموزشی متعادل‌تر شود.

۲. مدل فرصت تقریباً برابری برای مشاهده‌ی نمونه‌های کلاس‌های مختلف داشته باشد.

۳. عملکرد کلاس‌هایی که در حالت عادی ضعیف‌تر هستند، بهبود پیدا کند.

۴. اختلاف میان عملکرد کلاس‌های مختلف کاهش پیدا کند.

۵. در صورت موفق بودن روش، معیارهایی مانند Macro Recall و Macro F1 بهبود پیدا کنند.

۶. در مقابل، این احتمال نیز وجود دارد که دقت کلی مدل کاهش پیدا کند؛ زیرا مدل دیگر صرفاً به سمت کلاس‌های پرتعداد متمایل نیست.

بنابراین هدف این آزمایش صرفاً دستیابی به یک عدد بالاتر برای دقت نیست، بلکه بررسی این موضوع است که آیا می‌توان با متعادل کردن نمونه‌های هر Batch، مدل را به سمت یادگیری منصفانه‌تر و متعادل‌تر تمام کلاس‌های خودرو هدایت کرد یا خیر.

در ادامه، مدل را یک بار با روش معمول و یک بار با استفاده از Balanced Batch Sampling آموزش می‌دهیم و سپس نتایج حاصل از این دو حالت را با یکدیگر مقایسه خواهیم کرد.

نتایج آموزش با Batch Balanced

پس از آموزش مدل با استفاده از Balanced Batch Sampling، روند تغییرات دقت آموزش و اعتبارسنجی مطابق شکل مربوط به این آزمایش مشاهده شد.

همان‌طور که در نمودار مشاهده می‌شود، دقت آموزش مدل در چند Epoch ابتدایی به سرعت افزایش پیدا کرده و پس از حدود ۹۸.۰ تا ۹۹.۰ رسیده است. پس از آن، دقت آموزش در همین محدوده باقی مانده و نوسانات نسبتاً کوچکی داشته است.

دقت اعتبارسنجی نیز در ابتدای آموزش افزایش قابل توجهی دارد و پس از چند Epoch به محدوده‌ی حدود ۸۵.۰ تا ۹۰.۰ می‌رسد. در ادامه، مقدار آن با نوسانات مختلف در اطراف ۹۰.۰ باقی می‌ماند. بهترین Epoch مشخص شده در نمودار، Epoch شماره‌ی ۶۹ است که در آن بهترین وضعیت اعتبارسنجی در فرآیند انتخاب مدل ثبت شده است.

نکته‌ی قابل توجه این است که پس از رسیدن مدل به دقت آموزش بسیار بالا، فاصله‌ای میان دقت آموزش و اعتبارسنجی ایجاد می‌شود. این فاصله نشان می‌دهد که مدل عملکرد بسیار خوبی روی داده‌های آموزشی دارد، اما عملکرد آن روی داده‌های اعتبارسنجی به همان اندازه افزایش پیدا نکرده است. بنابراین می‌توان مشاهده کرد که در ادامه‌ی آموزش، افزایش دقت آموزش الزاماً به افزایش دقت اعتبارسنجی منجر نشده است.

مقایسه با آموزش معمول

برای بررسی تأثیر Balanced Batch Sampling، نتایج آن را با آموزش مدل در حالت معمول مقایسه می‌کنیم.

در حالت معمول، همان‌طور که در نمودار مربوط به آموزش بدون Balanced Batch مشاهده می‌شود، دقت آموزش نیز در چند Epoch ابتدایی به سرعت افزایش پیدا کرده و در ادامه به محدوده‌ی بسیار بالایی، نزدیک به ۹۹.۰، رسیده است. دقت اعتبارسنجی در این حالت نیز پس از افزایش سریع اولیه، عمدتاً در محدوده‌ی ۹۰.۰ تا ۹۲.۰ قرار گرفته است.

در مقایسه‌ی دو نمودار، یکی از تفاوت‌های اصلی مربوط به بهترین Epoch ثبت شده است. در حالت Balanced Batch، بهترین Epoch در حدود ۶۹ قرار گرفته است، در حالی که در حالت معمول بهترین Epoch در حدود ۱۷۲ ثبت شده است. این موضوع نشان می‌دهد که رفتار مدل در فرآیند آموزش و زمان رسیدن به بهترین عملکرد اعتبارسنجی در دو روش متفاوت بوده است.

از نظر دقت آموزش، هر دو روش عملکرد بسیار بالایی دارند و دقت آموزش آن‌ها در بخش عمده‌ای از فرآیند آموزش به

محدوده‌ی ۹۸.۰ تا ۹۹.۰ نزدیک می‌شود. بنابراین تفاوت اصلی دو روش را نمی‌توان صرفاً از روی دقت آموزش مشاهده کرد.

در مقابل، دقت اعتبارسنجی در حالت معمول به طور کلی کمی بالاتر از حالت Balanced Batch قرار دارد. در حالت معمول، دقت اعتبارسنجی در بسیاری از Epochها در محدوده‌ی حدود ۹۰.۰ تا ۹۲.۰ قرار گرفته و در برخی نقاط به حدود ۹۳.۰ نیز نزدیک می‌شود. در حالت Balanced Batch، دقت اعتبارسنجی عمدتاً در محدوده‌ی حدود ۸۸.۰ تا ۹۱.۰ قرار دارد.

بنابراین بر اساس نمودارهای دقت، استفاده از Balanced Batch در این آزمایش باعث افزایش مستقیم Validation Accuracy نشده است. حتی می‌توان مشاهده کرد که بهترین عملکرد اعتبارسنجی در حالت معمول اندکی بالاتر از حالت Balanced Batch بوده است.

با این حال، این نتیجه به تنهایی به معنی ناموفق بودن Balanced Batch نیست. دلیل این موضوع آن است که هدف اصلی Batch Balanced افزایش دقت کلی نیست، بلکه کاهش تأثیر عدم توازن کلاس‌ها و ایجاد فرصت یادگیری متعادل‌تر برای کلاس‌های مختلف است.

تحلیل اختلاف بین Accuracy Validation و Training

در هر دو آزمایش، دقت آموزش به مقدار بسیار بالایی می‌رسد، در حالی که دقت اعتبارسنجی پایین‌تر باقی می‌ماند. بنابراین در هر دو حالت فاصله‌ای میان Training Accuracy و Validation Accuracy مشاهده می‌شود.

این فاصله می‌تواند نشان‌دهنده‌ی وجود مقداری Overfitting باشد. به عبارت دیگر، مدل توانسته است الگوهای موجود در داده‌های آموزشی را با دقت بسیار بالایی یاد بگیرد، اما این افزایش عملکرد به همان میزان به داده‌های اعتبارسنجی منتقل نشده است.

در حالت Balanced Batch نیز با وجود اینکه مدل در داده‌های آموزشی به دقت نزدیک به ۹۹.۰ می‌رسد، دقت اعتبارسنجی عمدتاً در محدوده‌ی ۹۰.۰ تا ۹۱.۰ باقی می‌ماند. بنابراین متعادل کردن Batchها به تنهایی باعث حذف کامل فاصله‌ی میان عملکرد آموزش و اعتبارسنجی نشده است.

از طرف دیگر، نوسانات دقت اعتبارسنجی در هر دو نمودار نشان می‌دهد که عملکرد مدل روی داده‌های اعتبارسنجی کاملاً ثابت نیست. این نوسانات در حالت Balanced Batch نیز وجود دارند و بنابراین نمی‌توان صرفاً بر اساس افزایش یا کاهش یک Epoch خاص درباره‌ی برتری یکی از روش‌ها نتیجه‌گیری کرد.

جمع‌بندی مقایسه‌ی دو روش

بر اساس نمودارهای به‌دست‌آمده، می‌توان نتایج اولیه‌ی دو روش را به صورت زیر خلاصه کرد:

۱. هر دو روش توانسته‌اند دقت آموزش بسیار بالایی، در حدود ۹۸.۰ تا ۹۹.۰، ایجاد کنند.

۲. دقت اعتبارسنجی در حالت معمول به طور کلی کمی بالاتر از حالت Balanced Batch قرار دارد.

۳. بهترین Epoch در حالت Balanced Batch حدود ۶۹ بوده است، در حالی که در حالت معمول حدود Epoch ۱۷۲ ثبت شده است.

۴. استفاده از Balanced Batch در این آزمایش باعث افزایش مستقیم Validation Accuracy نشده است.

۵. در هر دو روش فاصله‌ی مشخصی میان Training Accuracy و Validation Accuracy وجود دارد که می‌تواند نشان‌دهنده‌ی مقداری Overfitting باشد.

۶. با وجود پایین‌تر بودن دقت اعتبارسنجی در حالت Balanced Batch، نمی‌توان بر اساس Accuracy به تنهایی درباره‌ی موفقیت یا شکست این روش قضاوت کرد.

۷. برای بررسی هدف اصلی Balanced Batch، یعنی ایجاد عملکرد متعادل‌تر میان کلاس‌ها، باید معیارهایی مانند Macro Precision، Macro Recall و Macro F1 و همچنین نتایج مربوط به هر یک از ۸ کلاس بررسی شوند.

در نتیجه، نتایج حاصل از نمودارها نشان می‌دهند که در این آزمایش، Balanced Batch Sampling نسبت به روش معمول مزیت مشخصی از نظر افزایش Validation Accuracy ایجاد نکرده است. با این حال، از آنجا که هدف اصلی این روش کاهش تأثیر عدم توازن کلاس‌ها است، برای ارزیابی نهایی باید بررسی کنیم که آیا عملکرد کلاس‌های کم‌تعداد بهبود یافته و آیا معیارهای میانگین‌گیری‌شده بر اساس کلاس، به‌خصوص Macro F1 و Macro Recall، افزایش پیدا کرده‌اند یا خیر.

بنابراین می‌توان نتیجه گرفت که نمودارهای Accuracy تنها بخشی از اثر Balanced Batch Sampling را نشان می‌دهند و برای یک نتیجه‌گیری نهایی، مقایسه‌ی عملکرد تک‌تک کلاس‌ها و معیارهای Macro ضروری است.

آزمایش Weight Decay

پس از بررسی تأثیر Balanced Batch Sampling، در آزمایش بعدی به سراغ یکی دیگر از روش‌های مهم برای کنترل Overfitting می‌رویم. این روش Weight Decay نام دارد.

همان‌طور که در آزمایش CustomCNN مشاهده کردیم، مدل توانست دقت بسیار بالایی روی داده‌های آموزشی به دست آورد، اما دقت اعتبارسنجی با فاصله‌ی قابل توجهی پایین‌تر باقی ماند. این اختلاف میان عملکرد آموزش و اعتبارسنجی می‌تواند نشانه‌ای از Overfitting باشد.

یکی از روش‌هایی که می‌توان برای کاهش این مشکل استفاده کرد، اعمال جریمه روی وزن‌های بزرگ شبکه است. Weight Decay دقیقاً با همین هدف وارد فرآیند آموزش می‌شود.

Weight Decay چیست؟

در هنگام آموزش شبکه عصبی، هدف الگوریتم این است که پارامترهای مدل را به گونه‌ای تنظیم کند که خطای پیش‌بینی کاهش پیدا کند.

در حالت معمول، تابع هزینه تنها بر اساس خطای پیش‌بینی مدل تعریف می‌شود. اما در صورت استفاده از Weight Decay، یک جمله‌ی جریمه نیز به تابع هزینه اضافه می‌شود که مقدار وزن‌های شبکه را کنترل می‌کند.

به صورت ساده، تابع هدف را می‌توان به شکل زیر در نظر گرفت:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \sum_i w_i^2$$

که در آن:

- $\mathcal{L}_{data}$ خطای مربوط به پیش‌بینی مدل است.

- w_i وزن‌های قابل آموزش شبکه هستند.

- λ ضریب Weight Decay است.

در این آزمایش مقدار:

$$\lambda = 10^{-4}$$

در نظر گرفته شده است.

یعنی مقدار Weight Decay برابر با:

$$10^{-4} = 0.0001$$

است.

چرا از Weight Decay استفاده می‌کنیم؟

ایده‌ی اصلی Weight Decay این است که مدل را از یادگیری وزن‌های بسیار بزرگ و پیچیده شدن بیش از حد آن بازدارد. اگر هیچ محدودیتی روی وزن‌های شبکه وجود نداشته باشد، مدل ممکن است برای کاهش خطای داده‌های آموزشی، وزن‌های خود را به مقادیر بزرگی برساند و الگوهای بسیار خاص موجود در داده‌های آموزش را یاد بگیرد.

در چنین شرایطی ممکن است Training Accuracy بسیار بالا باشد، اما مدل روی تصاویر جدید عملکرد ضعیف‌تری داشته باشد. این همان مسئله‌ای است که با عنوان Overfitting شناخته می‌شود.

با اضافه کردن جمله‌ی جریمه‌ی Weight Decay، مدل علاوه بر تلاش برای کاهش خطای پیش‌بینی، باید اندازه‌ی وزن‌های خود را نیز در یک محدوده‌ی مناسب نگه دارد.

به بیان ساده، مدل با یک نوع جریمه مواجه می‌شود:

$$\text{جریمه‌ی بیشتر} \implies \text{وزن‌های بزرگ‌تر}$$

بنابراین به جای اینکه صرفاً تلاش کند داده‌های آموزشی را تا حد ممکن حفظ کند، تشویق می‌شود که راه‌حل ساده‌تر و قابل تعمیم‌تری پیدا کند.

تأثیر مقدار 10^{-4}

مقدار Weight Decay نقش مهمی در میزان Regularization دارد.

اگر مقدار λ بسیار کوچک باشد، اثر جریمه ضعیف خواهد بود و ممکن است تأثیر چندانی بر رفتار مدل نداشته باشد.

از طرف دیگر، اگر مقدار آن بیش از حد بزرگ انتخاب شود، جریمه‌ی اعمال‌شده به وزن‌ها می‌تواند بسیار شدید شود و مانع از یادگیری مناسب مدل شود.

در این آزمایش مقدار:

$$\lambda = 10^{-4}$$

انتخاب شده است.

این مقدار یک Regularization نسبتاً ملایم ایجاد می‌کند و هدف این است که بدون جلوگیری از یادگیری ویژگی‌های مهم، از پیچیده شدن بیش از حد مدل جلوگیری شود.

Optimizer در Weight Decay

در این پروژه، Weight Decay به صورت مستقیم در فرآیند بهینه‌سازی و همراه با Adam اعمال می‌شود. بنابراین در آزمایش جدید، علاوه بر تنظیمات قبلی، پارامتر Weight Decay برابر با 10^{-4} قرار داده شده است. به صورت خلاصه:

Optimizer: Adam

Learning Rate: 10^{-4}

Weight Decay: 10^{-4}

هدف: کاهش Overfitting و افزایش توانایی تعمیم مدل

چه انتظاری از این آزمایش داریم؟

انتظار داریم اضافه کردن Weight Decay باعث شود مدل کمتر به جزئیات خاص داده‌های آموزشی وابسته شود. در نتیجه، ممکن است Training Accuracy کمی کاهش پیدا کند. این کاهش لزوماً اتفاق بدی نیست؛ زیرا هدف اصلی این آزمایش دستیابی به بیشترین دقت ممکن روی داده‌های آموزش نیست. در مقابل، انتظار داریم در صورت انتخاب مناسب مقدار Weight Decay، فاصله‌ی میان Training Accuracy و Validation Accuracy کاهش پیدا کند و عملکرد مدل روی داده‌های دیده‌نشده بهتر شود. بنابراین معیار مهم برای مقایسه، تنها افزایش Training Accuracy نیست، بلکه باید بررسی کنیم که آیا Validation Accuracy پایدارتر یا بهتر شده است و آیا نشانه‌های Overfitting کاهش پیدا کرده‌اند یا خیر. به طور خلاصه، انتظار داریم:

۱. دقت آموزش ممکن است کمی کاهش پیدا کند.

۲. مدل کمتر به نمونه‌های آموزشی وابسته شود.

۳. فاصله‌ی میان دقت آموزش و اعتبارسنجی کاهش پیدا کند.

۴. دقت اعتبارسنجی در صورت مناسب بودن مقدار Weight Decay افزایش پیدا کند یا پایدارتر شود.

۵. توانایی تعمیم مدل روی تصاویر جدید بهبود پیدا کند.

در ادامه، مدل را با 10^{-4} = Weight Decay آموزش می‌دهیم و نمودارهای Training Accuracy و Validation Accuracy را با نتایج آزمایش‌های قبلی مقایسه خواهیم کرد.

نتایج آزمایش Weight Decay

پس از آموزش مدل با استفاده از Weight Decay برابر با 10^{-4} ، روند تغییرات دقت آموزش و اعتبارسنجی در شکل زیر نشان داده شده است.

همان‌طور که در شکل مشاهده می‌شود، ...

آزمایش Learning Rate Scheduler

در این آزمایش برای تغییر مرحله‌ای نرخ یادگیری از StepLR استفاده شد. نرخ یادگیری اولیه برابر با 10^{-4} بود و پس از هر ۵۰ Epoch با ضریب ۵.۰ کاهش یافت.

تنظیمات StepLR

پارامترهای Scheduler به صورت زیر تنظیم شدند:

$$\text{step_size} = 50$$

$$\gamma = 5.0$$

رابطه‌ی تغییر نرخ یادگیری:

$$LR_{\text{new}} = LR_{\text{old}} \times 0.5.$$

بنابراین نرخ یادگیری در طول آموزش به صورت زیر تغییر می‌کند:

Rate Learning	بازه‌ی Epoch
1e-4	1 – 50
5e-5	51 – 100
2.5e-5	101 – 150
1.25e-5	151 – 200
6.25e-6	201 – 250
3.125e-6	251 – 300
1.5625e-6	301 – 350
7.8125e-7	351 – 400
3.90625e-7	401 – 450
1.953125e-7	451 – 500

تنظیمات آزمایش

در این آزمایش همان معماری CustomCNN استفاده شد. مقدار Seed برابر با ۴۲، تعداد کلاس‌ها برابر با ۸ و بهینه‌ساز Adam در نظر گرفته شد.

Seed: ۴۲
 Model: CustomCNN
 Number of Classes: ۸
 Optimizer: Adam
 Initial Learning Rate: 10^{-4}
 Scheduler: StepLR
 Step Size: ۵۰
 Gamma: ۰.۵
 Maximum Epochs: ۵۰۰
 Early Stopping Patience: ۷۵

نتایج آزمایش Learning Rate Scheduler

مدل آموزش را با نرخ یادگیری $1e-4$ آغاز کرد. دقت آموزش در Epoch اول برابر با 51.71% و در Epoch پنجم برابر با 89.28% بود. در Epoch نهم، دقت آموزش از 96% عبور کرد.

دقت اعتبارسنجی از 68.69% در Epoch اول به 89.67% در Epoch نهم رسید.

در پایان Epoch 50، نرخ یادگیری از $1e-4$ به $5e-5$ کاهش یافت. دقت اعتبارسنجی در Epoch 51 برابر با 91.25% و در Epoch 66 برابر با 92.03% بود.

در Epoch 100 نرخ یادگیری به $2.5e-5$ کاهش یافت و دقت اعتبارسنجی در Epoch 108 به 92.37% رسید.
در Epoch 150 نرخ یادگیری به $1.25e-5$ کاهش یافت. بهترین دقت اعتبارسنجی در Epoch های ۱۵۴ و ۱۶۶ برابر با:

$$92.82\%$$

بود.

در Epoch 154، دقت آموزش برابر با 99.97% و دقت اعتبارسنجی برابر با 92.82% بود. اختلاف این دو مقدار برابر است با:

$$99.97\% - 92.82\% = 7.15\%$$

در ادامه، دقت اعتبارسنجی بالاتر از 92.82% ثبت نشد. در Epoch 201 نرخ یادگیری به $6.25e-6$ رسید، اما بهبود جدیدی در بهترین دقت اعتبارسنجی ایجاد نشد.

تأثیر Early Stopping

حداکثر تعداد Epoch برابر با 500 و مقدار patience برابر با 75 بود. فرآیند آموزش در Epoch 229 متوقف شد.

جمع‌بندی آزمایش

استفاده از StepLR باعث شد نرخ یادگیری از $1e-4$ شروع شده و پس از هر ۵۰ Epoch با ضریب 0.5 کاهش پیدا کند. بهترین دقت اعتبارسنجی برابر با 92.82% در Epoch های ۱۵۴ و ۱۶۶ ثبت شد. در این نقطه، دقت آموزش 99.97% بود و اختلاف آموزش و اعتبارسنجی برابر با 7.15% به دست آمد. در نهایت، آموزش در Epoch 229 توسط Early Stopping متوقف شد.

بهترین Validation Accuracy: 92.82%

Train Accuracy: 99.97%

تعداد Epoch اجراشده: 229

Initial Learning Rate: $1e-4$

Scheduler: StepLR

Step Size: 50

Gamma: 0.5

مقایسه‌ی Cross-Entropy و Binary Cross-Entropy

یکی دیگر از آزمایش‌های انجام‌شده در این پروژه، بررسی تأثیر تابع هزینه بر فرآیند یادگیری شبکه است.

تابع هزینه یکی از اجزای مهم فرآیند آموزش شبکه‌ی عصبی است و مشخص می‌کند که پیش‌بینی‌های مدل تا چه اندازه با برچسب واقعی نمونه‌ها اختلاف دارند. در این آزمایش، عملکرد دو تابع هزینه متفاوت با یکدیگر مقایسه شد:

۱. Cross-Entropy

۲. Binary Cross-Entropy (BCE)

هدف از این آزمایش، بررسی این موضوع بود که کدام تابع هزینه می‌تواند در مسئله‌ی طبقه‌بندی تصاویر خودروهای ترافیکی، عملکرد بهتری ایجاد کند.

Cross-Entropy چیست؟

تابع Cross-Entropy یکی از توابع هزینه رایج برای مسائل طبقه‌بندی چندکلاسه است. در این پروژه، مسئله شامل ۸ کلاس مختلف است و هر تصویر تنها به یکی از این کلاس‌ها تعلق دارد. بنابراین از نظر ماهیت مسئله، استفاده از Cross-Entropy یک انتخاب طبیعی برای طبقه‌بندی چندکلاسه محسوب می‌شود. این تابع اختلاف میان توزیع احتمال پیش‌بینی‌شده توسط مدل و برچسب واقعی نمونه را اندازه‌گیری می‌کند. برای یک نمونه، تابع هزینه Cross-Entropy به صورت کلی برابر است با:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

که در آن:

C

تعداد کلاس‌ها،

y_i

برچسب واقعی کلاس و

$\hat{y}_i$

احتمال پیش‌بینی‌شده برای کلاس مورد نظر است.

در این پروژه مقدار:

$$C = 8$$

است.

بنابراین مدل باید در هر مرحله احتمال مربوط به ۸ کلاس را محاسبه کرده و احتمال بیشتری را به کلاس صحیح اختصاص دهد.

Binary Cross-Entropy چیست؟

تابع Binary Cross-Entropy یا BCE در حالت استاندارد برای مسائل دودویی استفاده می‌شود.

فرمول کلی این تابع به صورت زیر است:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

در این روش، مدل برای هر خروجی میزان اطمینان خود را نسبت به یک حالت دودویی مشخص می‌کند.

در مسائل چندکلاسه نیز می‌توان از این رویکرد در قالب چند خروجی مستقل استفاده کرد. در این حالت، هر خروجی می‌تواند میزان تعلق نمونه به یک کلاس مشخص را نشان دهد.

در این پروژه، عملکرد این روش به صورت تجربی با Cross-Entropy مقایسه شد تا مشخص شود کدام روش روی مجموعه داده‌ی مورد استفاده عملکرد بهتری ایجاد می‌کند.

تنظیمات آزمایش

در هر دو آزمایش، هدف این بود که سایر شرایط آموزش تا حد امکان یکسان باقی بمانند و تنها تابع هزینه تغییر کند. بنابراین تفاوت اصلی مورد بررسی، استفاده از دو تابع هزینه مختلف بود.

تابع هزینه‌ی اول: Cross-Entropy

تابع هزینه‌ی دوم: Binary Cross-Entropy (BCE)

تعداد کلاس‌ها: ۸

معیار بررسی: Training Accuracy و Validation Accuracy

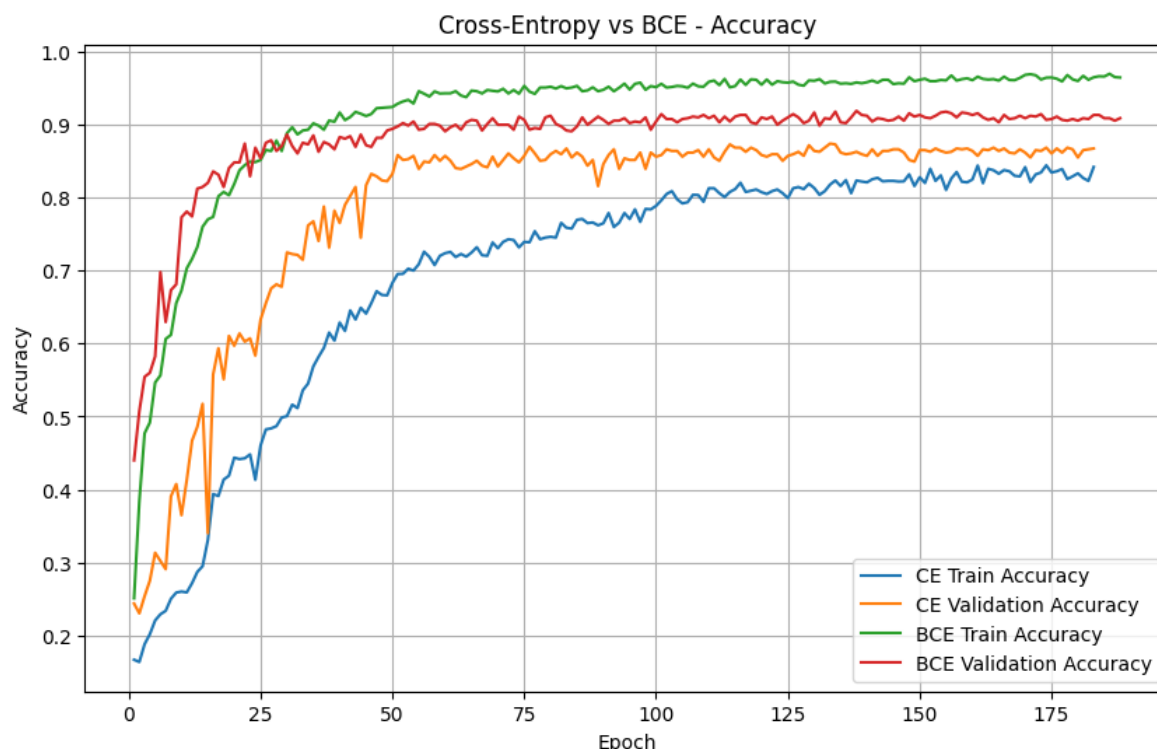
هدف اصلی این آزمایش افزایش صرف دقت آموزش نبود، بلکه بررسی توانایی هر تابع هزینه در یادگیری ویژگی‌های مناسب و تعمیم آن‌ها به داده‌های اعتبارسنجی بود.

نتایج آزمایش

روند تغییرات دقت آموزش و اعتبارسنجی برای هر دو تابع هزینه در شکل زیر نشان داده شده است.

همان‌طور که در شکل مشاهده می‌شود، رفتار دو تابع هزینه در طول فرآیند آموزش تفاوت قابل توجهی دارد.

در آزمایش Cross-Entropy، دقت آموزش در ابتدای فرآیند از مقدار نسبتاً پایینی شروع شده و به تدریج افزایش پیدا می‌کند. در چند Epoch ابتدایی، روند رشد دقت آموزش نسبتاً آرام است و سپس با ادامه‌ی آموزش، دقت افزایش پیدا می‌کند.



شکل ۲: مقایسه‌ی روند دقت آموزش و اعتبارسنجی در دو آزمایش Cross-Entropy و Binary Cross-Entropy.

در مقابل، دقت اعتبارسنجی Cross-Entropy با سرعت بیشتری در مراحل ابتدایی افزایش پیدا کرده و پس از حدود چند ده Epoch به محدوده‌ی تقریباً ۸۵.۰ تا ۸۸.۰ می‌رسد.

در بخش پایانی آموزش، دقت آموزش Cross-Entropy در حدود ۸۳.۰ تا ۸۵.۰ قرار می‌گیرد، در حالی که دقت اعتبارسنجی در محدوده‌ی تقریباً ۸۶.۰ تا ۸۸.۰ باقی می‌ماند.

بنابراین در آزمایش Cross-Entropy، برخلاف بسیاری از آزمایش‌های قبلی، فاصله‌ی بزرگی میان دقت آموزش و اعتبارسنجی مشاهده نمی‌شود.

حتی در بخش پایانی، دقت اعتبارسنجی در بسیاری از نقاط کمی بالاتر از دقت آموزش قرار دارد.

این موضوع نشان می‌دهد که در این آزمایش، مسئله‌ی اصلی Cross-Entropy بیش‌برازش شدید مدل روی داده‌های آموزش نبوده است.

نتایج Binary Cross-Entropy

رفتار Binary Cross-Entropy تفاوت قابل توجهی با Cross-Entropy دارد.

در ابتدای آموزش، دقت آموزش مدل با استفاده از BCE از حدود ۴۴.۰ شروع می‌شود و با سرعت زیادی افزایش پیدا می‌کند.

دقت آموزش در حدود Epoch‌های ۲۰ تا ۳۰ به محدوده‌ی بالاتر از ۸۵.۰ می‌رسد و سپس به تدریج به مقادیر بالاتر نزدیک می‌شود.

در ادامه‌ی آموزش، دقت آموزش BCE تقریباً به محدوده‌ی ۹۵.۰ تا ۹۷.۰ می‌رسد.

در مقابل، دقت اعتبارسنجی نیز در ابتدای آموزش رشد قابل توجهی دارد و در ادامه به محدوده‌ی حدود ۹۰.۰ تا ۹۲.۰

می‌رسد.

بنابراین، از نظر دقت نهایی، BCE توانسته است عملکرد بالاتری نسبت به Cross-Entropy ایجاد کند. به صورت تقریبی، در بخش پایانی آموزش:

$$\text{BCE Train Accuracy} \approx 96.0$$

9

$$\text{BCE Validation Accuracy} \approx 91.0$$

است.

در نتیجه، عملکرد BCE روی مجموعه‌ی اعتبارسنجی حدود چند واحد درصد بهتر از عملکرد Cross-Entropy قرار گرفته است.

مقایسه‌ی مستقیم دو تابع هزینه

با مقایسه‌ی چهار منحنی موجود در نمودار، تفاوت میان دو روش به خوبی قابل مشاهده است.

در روش Cross-Entropy، دقت آموزش در انتهای آزمایش تقریباً در محدوده‌ی ۸۴.۰ قرار دارد و دقت اعتبارسنجی به حدود ۸۷.۰ می‌رسد.

در مقابل، در روش BCE، دقت آموزش به حدود ۹۶.۰ می‌رسد و دقت اعتبارسنجی در محدوده‌ی حدود ۹۱.۰ قرار می‌گیرد. بنابراین می‌توان مقادیر تقریبی زیر را برای مقایسه در نظر گرفت:

روش	Accuracy Training	Accuracy Validation
Cross-Entropy	حدود ۸۴٪	حدود ۸۷٪
BCE	حدود ۹۶٪	حدود ۹۱٪

بر اساس این مقایسه، BCE از نظر دقت اعتبارسنجی عملکرد بهتری نسبت به Cross-Entropy داشته است. اختلاف تقریبی دقت اعتبارسنجی دو روش برابر است با:

$$91\% - 87\% \approx 4\%$$

بنابراین در این آزمایش، BCE توانسته است حدود ۴ واحد درصد دقت اعتبارسنجی بالاتری نسبت به Cross-Entropy ایجاد کند.

البته این اختلاف باید به صورت تقریبی در نظر گرفته شود، زیرا مقادیر فوق از روی نمودار استخراج شده‌اند.

مقایسه‌ی سرعت یادگیری

یکی از تفاوت‌های مهم دیگر میان دو روش، سرعت افزایش دقت در مراحل ابتدایی آموزش است. در Cross-Entropy، دقت آموزش در ابتدا مقدار پایینی دارد و سپس به صورت تدریجی افزایش پیدا می‌کند. در مقابل، BCE از همان Epoch‌های ابتدایی رشد سریع‌تری نشان می‌دهد. برای مثال، در ابتدای آموزش دقت BCE از حدود ۴۴٪ شروع شده و در چند ده Epoch نخست به بیش از ۸۰٪ می‌رسد. این در حالی است که دقت آموزش Cross-Entropy با سرعت کمتری افزایش پیدا می‌کند. بنابراین می‌توان گفت که در این آزمایش، BCE علاوه بر رسیدن به دقت بالاتر، از نظر سرعت رسیدن به عملکرد مناسب نیز رفتار مطلوب‌تری داشته است.

بررسی بیش‌برازش

یکی از نکات مهم موجود در نمودار، تفاوت میان دقت آموزش و اعتبارسنجی در روش BCE است. در مراحل پایانی، دقت آموزش BCE تقریباً به ۹۶٪ می‌رسد، در حالی که دقت اعتبارسنجی در محدوده‌ی حدود ۹۱٪ قرار دارد. بنابراین اختلاف تقریبی میان این دو مقدار برابر است با:

$$۹۶\% - ۹۱\% \approx ۵\%$$

این فاصله نشان می‌دهد که مدل در روش BCE تا حدی داده‌های آموزش را بهتر از داده‌های اعتبارسنجی یاد گرفته است.

در نتیجه، در روش BCE نشانه‌هایی از Overfitting مشاهده می‌شود. با این حال، نکته‌ی مهم این است که وجود مقداری بیش‌برازش لزوماً به معنی عملکرد نامناسب مدل نیست. در این آزمایش، با وجود این فاصله، دقت اعتبارسنجی BCE همچنان بالاتر از Cross-Entropy قرار گرفته است. بنابراین اگر معیار اصلی انتخاب تابع هزینه، عملکرد مدل روی داده‌های اعتبارسنجی باشد، نتایج این آزمایش به نفع BCE است.

تحلیل رفتار Cross-Entropy

رفتار Cross-Entropy نکته‌ی جالب دیگری را نشان می‌دهد. در این روش، دقت آموزش در انتهای آموزش تقریباً ۸۴٪ است، در حالی که دقت اعتبارسنجی به حدود ۸۷٪ می‌رسد. بنابراین:

$$\text{Validation Accuracy} > \text{Training Accuracy}$$

در بخشی از روند آموزش مشاهده می‌شود.

این حالت الزاماً نشان‌دهنده‌ی عملکرد بهتر مدل روی داده‌های ناشناخته نیست.

یکی از دلایل ممکن برای چنین رفتاری می‌تواند وجود تفاوت میان شرایط داده‌های آموزش و اعتبارسنجی باشد. برای مثال، اگر در مجموعه‌ی آموزش از روش‌های افزایش داده استفاده شده باشد، نمونه‌های آموزشی ممکن است دشوارتر از نمونه‌های اعتبارسنجی باشند.

همچنین نوسانات موجود در منحنی اعتبارسنجی نشان می‌دهد که عملکرد مدل در طول آموزش کاملاً پایدار نبوده است. در نتیجه، نمی‌توان صرفاً بر اساس بالاتر بودن موقت دقت اعتبارسنجی از دقت آموزش، نتیجه گرفت که مدل بدون مشکل عمل می‌کند.

چرا BCE در این آزمایش بهتر عمل کرده است؟

نتایج آزمایش نشان می‌دهند که BCE در این مجموعه داده توانسته است دقت بالاتری ایجاد کند.

با این حال، از روی نمودار به تنهایی نمی‌توان علت قطعی این برتری را مشخص کرد.

یکی از احتمالات این است که نحوه‌ی تبدیل خروجی‌های مدل به احتمال کلاس‌ها و شیوه‌ی محاسبه‌ی گرادیان در دو تابع هزینه متفاوت است.

در Cross-Entropy، خروجی‌های مدل معمولاً به صورت یک توزیع احتمال میان کلاس‌های رقیب در نظر گرفته می‌شوند. در مقابل، BCE می‌تواند هر خروجی را به صورت یک تصمیم مستقل بررسی کند.

بنابراین نحوه‌ی ایجاد گرادیان و جهت حرکت وزن‌های شبکه در دو روش متفاوت خواهد بود.

از آنجا که نتیجه‌ی آزمایش نشان داده است که BCE روی مجموعه داده‌ی فعلی عملکرد اعتبارسنجی بهتری ایجاد کرده است، می‌توان این روش را در ادامه‌ی پروژه به عنوان یکی از گزینه‌های مناسب برای آزمایش‌های بعدی در نظر گرفت.

با این حال، برای انتخاب نهایی تابع هزینه بهتر است علاوه بر Accuracy، معیارهایی مانند Precision، Recall، F1-Score و عملکرد تک‌تک کلاس‌ها نیز بررسی شوند.

جمع‌بندی آزمایش

در این آزمایش، دو تابع هزینه‌ی Cross-Entropy و Binary Cross-Entropy با یکدیگر مقایسه شدند.

نتایج نشان دادند که BCE در این آزمایش عملکرد بهتری نسبت به Cross-Entropy داشته است.

دقت اعتبارسنجی Cross-Entropy در بخش پایانی آموزش در محدوده‌ی حدود ۸۷٪ قرار داشت، در حالی که دقت اعتبارسنجی BCE به حدود ۹۱٪ رسید.

بنابراین اختلاف تقریبی عملکرد اعتبارسنجی دو روش حدود ۴ واحد درصد بود.

همچنین BCE از نظر سرعت افزایش دقت آموزش نیز رفتار سریع‌تری نشان داد و دقت آموزش آن در ادامه‌ی فرآیند به حدود ۹۶٪ رسید.

با این حال، فاصله‌ی میان دقت آموزش و اعتبارسنجی در روش BCE نشان می‌دهد که مقداری بیش‌برازش وجود دارد. در مقابل، در روش Cross-Entropy فاصله‌ی میان دقت آموزش و اعتبارسنجی بسیار کمتر بود، اما سطح کلی دقت اعتبارسنجی نیز پایین‌تر باقی ماند.

بنابراین، بر اساس نتایج این آزمایش، اگر معیار اصلی انتخاب روش، Validation Accuracy باشد، BCE عملکرد بهتری

نسبت به Cross-Entropy نشان داده است.

نتیجه‌ی نهایی آزمایش:

در این آزمایش، Binary Cross-Entropy توانست نسبت به Cross-Entropy دقت اعتبارسنجی بالاتری ایجاد کند. دقت اعتبارسنجی BCE در حدود ۹۱٪ و دقت اعتبارسنجی Cross-Entropy در حدود ۸۷٪ قرار گرفت. بنابراین، در مجموعه‌داده‌ی مورد استفاده در این پروژه، BCE از نظر Validation Accuracy عملکرد بهتری نشان داد. با این وجود، وجود فاصله میان دقت آموزش و اعتبارسنجی BCE نشان می‌دهد که مقداری Overfitting همچنان وجود دارد و برای ارزیابی کامل‌تر، بررسی معیارهای Precision، Recall و F1-Score نیز ضروری است.

افزایش داده و روش‌های Data Augmentation

یکی از روش‌های مورد استفاده در این پروژه برای افزایش تنوع داده‌های آموزشی، استفاده از تکنیک‌های Data Augmentation است.

هدف از افزایش داده، ایجاد تغییرات کنترل‌شده در تصاویر آموزشی است تا مدل بتواند ویژگی‌های مهم تصاویر را بهتر یاد بگیرد و نسبت به تغییرات مختلف موجود در تصاویر مقاوم‌تر شود.

در این پروژه، چندین تکنیک مختلف افزایش داده به صورت جداگانه و ترکیبی مورد بررسی قرار گرفتند. اندازه‌ی نهایی تمام تصاویر ورودی برابر با ۱۵۰×۱۵۰ پیکسل در نظر گرفته شد.

روش‌های اصلی مورد استفاده در این پروژه عبارت‌اند از:

۱. Random Resized Crop

۲. Random Horizontal Flip

۳. Random Rotation

۴. Color Jitter

۵. Gaussian Blur

۶. Random Erasing

Random Resized Crop

در این روش، ابتدا بخشی تصادفی از تصویر انتخاب شده و سپس آن بخش به اندازه‌ی ۱۵۰×۱۵۰ تغییر اندازه داده می‌شود.

در این پروژه محدوده‌ی انتخاب بخش تصویر به صورت زیر تعیین شده است:

$$\text{scale} = 0.7 - 1.0$$

بنابراین بین 70% تا 100% از مساحت تصویر می‌تواند به صورت تصادفی انتخاب شده و سپس به اندازه‌ی مورد نظر تغییر داده شود.

این روش باعث می‌شود مدل نسبت به تغییرات مربوط به موقعیت و اندازه‌ی خودرو در تصویر مقاوم‌تر شود.

Random Horizontal Flip

در این روش تصویر با احتمال مشخصی به صورت افقی برگردانده می‌شود.

در این پروژه احتمال اعمال این تبدیل برابر با 0.5 در نظر گرفته شده است:

$$P(\text{Horizontal Flip}) = 0.5$$

بنابراین تقریباً در نیمی از نمونه‌های آموزشی، تصویر به صورت افقی معکوس می‌شود.

هدف از این تبدیل، افزایش تنوع جهت قرارگیری خودرو در تصاویر آموزشی است.

Random Rotation

در این روش تصویر به صورت تصادفی در محدوده‌ی مشخصی چرخانده می‌شود.

در این پروژه حداکثر زاویه‌ی چرخش برابر با 10 درجه در نظر گرفته شده است:

$$-10 - +10 \text{ درجه}$$

این تبدیل به مدل کمک می‌کند تا نسبت به تغییرات جزئی زاویه‌ی قرارگیری تصویر حساسیت کمتری داشته باشد.

Color Jitter

تکنیک Color Jitter برای ایجاد تغییرات تصادفی در ویژگی‌های رنگی تصویر استفاده می‌شود.

در این پروژه پارامترهای آن به صورت زیر تنظیم شده‌اند:

$$\text{brightness} = 0.3$$

$$\text{contrast} = 0.3$$

$$\text{saturation} = 0.3$$

$$\text{hue} = 0.1$$

بنابراین روشنایی، کنتراست، اشباع رنگ و تهرنگ تصویر می‌توانند به صورت تصادفی تغییر کنند.

هدف از این روش، افزایش مقاومت مدل در برابر تغییرات شرایط نوری و رنگی تصاویر است.

Gaussian Blur

در این روش، فیلتر Gaussian Blur برای ایجاد تاری کنترل‌شده در تصویر استفاده می‌شود. در این پروژه اندازه‌ی کرنل برابر با 3 و محدوده‌ی پارامتر sigma به صورت زیر در نظر گرفته شده است:

$$\text{kernel_size} = 3$$

$$\text{sigma} = 0.1 - 2.0$$

این تبدیل می‌تواند به مدل کمک کند تا وابستگی بیش از حد به جزئیات بسیار ریز تصویر کاهش پیدا کند.

Random Erasing

در روش Random Erasing، بخشی از تصویر به صورت تصادفی حذف یا پوشانده می‌شود. در این پروژه احتمال اعمال این تبدیل برابر با 0.3 است:

$$P(\text{Random Erasing}) = 0.3$$

و مساحت بخش حذف‌شده در محدوده‌ی زیر قرار دارد:

$$\text{scale} = 0.02 - 0.15$$

این روش باعث می‌شود مدل برای تشخیص کلاس تنها به یک بخش مشخص از تصویر وابسته نباشد و بتواند در شرایطی که بخشی از خودرو پوشانده شده است نیز عملکرد مناسبی داشته باشد.

ترکیب روش‌های افزایش داده

برای بررسی تأثیر میزان و نوع افزایش داده، تکنیک‌های بالا در چند ترکیب مختلف مورد استفاده قرار گرفتند. در مجموع، هشت حالت افزایش داده در آزمایش‌ها تعریف شد:

تعداد تکنیک‌ها	نوع افزایش داده
۰	No Augmentation
۲	Basic
۱	Color
۱	Crop
۳	Full without Color
۵	Full + Basic
۶	Full + Basic + Color
۴	Full

در حالت No Augmentation هیچ تبدیل تصادفی روی تصویر اعمال نمی‌شود و تنها تغییر اندازه و تبدیل تصویر به Tensor انجام می‌شود.

حالت Basic شامل دو تکنیک Random Horizontal Flip و Random Rotation است.

در حالت Color تنها از Color Jitter استفاده می‌شود و در حالت Crop تنها Random Resized Crop مورد استفاده قرار می‌گیرد.

حالت Full without Color شامل Random Resized Crop، Gaussian Blur و Random Erasing است و هیچ تغییر رنگی در آن اعمال نمی‌شود.

در حالت Full + Basic علاوه بر تکنیک‌های حالت Full without Color، دو تبدیل Random Horizontal Flip و Random Rotation نیز اضافه می‌شوند.

حالت Full + Basic + Color شامل تمام تکنیک‌های مورد استفاده در آزمایش به جز تفاوتی در ترکیب آن‌ها نیست و در آن Color Jitter نیز به مجموعه‌ی تبدیل‌ها اضافه می‌شود.

در نهایت، حالت Full شامل Random Resized Crop، Color Jitter، Gaussian Blur و Random Erasing است.

تبدیلات داده در مرحله‌ی ارزیابی

در مرحله‌ی ارزیابی و اعتبارسنجی، برای جلوگیری از ایجاد تغییرات تصادفی در تصاویر، هیچ‌یک از تکنیک‌های تصادفی افزایش داده استفاده نشد.

در این مرحله تنها اندازه‌ی تصاویر به 150×150 تغییر داده شده و سپس تصاویر به Tensor تبدیل شدند:

$$\text{Resize}(150, 150) \rightarrow \text{ToTensor}$$

این کار باعث می‌شود تمام مدل‌ها روی داده‌های ارزیابی یکسان و بدون تغییرات تصادفی مورد ارزیابی قرار گیرند.

یادگیری انتقالی و استفاده از ResNet18

پس از بررسی و ارزیابی شبکه‌ی عصبی سفارشی، در مرحله‌ی بعد از یک معماری ازپیش‌آموزته‌شده به نام ResNet18 استفاده شد.

هدف از این آزمایش، بررسی این موضوع است که آیا استفاده از ویژگی‌هایی که یک شبکه‌ی عمیق از قبل یاد گرفته است می‌تواند عملکرد طبقه‌بندی تصاویر خودروهای ترافیکی را بهبود دهد یا خیر.

در این روش، به جای آنکه تمام پارامترهای شبکه از ابتدا آموزش داده شوند، از وزن‌های ازپیش‌آموزته‌شده‌ی شبکه استفاده می‌شود و تنها بخش‌هایی از شبکه که برای مسئله‌ی جدید مورد نیاز هستند، مجدداً آموزش داده می‌شوند.

این روش با عنوان Transfer Learning شناخته می‌شود.

چرا از Transfer Learning استفاده می‌کنیم؟

آموزش یک شبکه‌ی عصبی عمیق از ابتدا معمولاً به حجم زیادی از داده و زمان آموزشی قابل توجهی نیاز دارد. در مقابل، شبکه‌هایی مانند ResNet18 قبلاً روی مجموعه‌داده‌های بزرگ آموزش دیده‌اند و در طول آموزش خود ویژگی‌های مختلفی از تصاویر را یاد گرفته‌اند. لایه‌های ابتدایی شبکه‌های کانولوشنی معمولاً ویژگی‌های عمومی‌تری مانند لبه‌ها، تغییرات شدت روشنایی، بافت‌ها و الگوهای ساده را استخراج می‌کنند. با حرکت به سمت لایه‌های عمیق‌تر، ویژگی‌ها پیچیده‌تر و تخصصی‌تر می‌شوند. بنابراین می‌توان از بخش زیادی از دانش یادگرفته‌شده‌ی شبکه‌ی ازپیش‌آموزش‌شده استفاده کرد و تنها بخش‌های مورد نیاز را برای مسئله‌ی جدید آموزش داد. در پروژه‌ی حاضر، هدف این است که ویژگی‌های استخراج‌شده توسط ResNet18 برای شناسایی کلاس‌های مختلف خودروهای ترافیکی مورد استفاده قرار گیرند.

معماری ResNet18

ResNet18 یکی از معماری‌های شناخته‌شده در حوزه‌ی شبکه‌های عصبی کانولوشنی است. ویژگی مهم خانواده‌ی ResNet استفاده از اتصال‌های میان‌بر یا Skip Connection است. در یک شبکه‌ی عمیق معمولی، خروجی هر لایه مستقیماً به لایه‌ی بعدی منتقل می‌شود. اما در ResNet، بخشی از ورودی می‌تواند از چند لایه عبور کرده و مستقیماً به خروجی یک بخش عمیق‌تر اضافه شود. به صورت ساده می‌توان این مفهوم را به شکل زیر نمایش داد:

$$y = F(x) + x$$

در این رابطه، x ورودی بلوک، $F(x)$ تبدیل انجام‌شده توسط لایه‌های داخل بلوک و y خروجی بلوک است. وجود این اتصال‌های میان‌بر باعث می‌شود آموزش شبکه‌های عمیق ساده‌تر و پایدارتر شود. در معماری ResNet18، شبکه از چند بخش متوالی تشکیل شده است که در هر مرحله ویژگی‌های پیچیده‌تری از تصویر استخراج می‌شوند. در این پروژه، خروجی نهایی شبکه برای مسئله‌ی طبقه‌بندی 8 کلاسه تنظیم شده است. بنابراین لایه‌ی طبقه‌بندی نهایی باید بتواند تصویر ورودی را در یکی از 8 کلاس موجود در مجموعه‌داده قرار دهد.

راهبرد استفاده از Transfer Learning

در این آزمایش از معماری ازپیش‌آموزش‌شده‌ی ResNet18 استفاده شد. مدل با استفاده از وزن‌های ازپیش‌آموزش‌شده ایجاد شده است. سپس لایه‌ی طبقه‌بندی نهایی شبکه متناسب با مسئله‌ی حاضر تغییر داده شد تا خروجی مدل شامل 8 کلاس باشد. در ادامه، به منظور جلوگیری از تغییر غیرضروری ویژگی‌های عمومی یادگرفته‌شده توسط شبکه، تمام پارامترهای مدل در ابتدا فریز شدند و سپس تنها دو بخش مشخص از شبکه برای آموزش مجدداً فعال شدند:

• layer4

• fc

بنابراین ساختار آموزش مدل به صورت زیر در نظر گرفته شد:

conv1, bn1, layer1, layer2, layer3

→

فریز شده

:9

layer4

→

قابل آموزش

:9

fc

→

قابل آموزش

در نتیجه، بخش‌های ابتدایی شبکه که وظیفه‌ی استخراج ویژگی‌های عمومی را بر عهده دارند، در طول آموزش این آزمایش تغییر نمی‌کنند.

در مقابل، layer4 که آخرین بخش اصلی استخراج ویژگی در شبکه است، امکان یادگیری و سازگار شدن با ویژگی‌های خاص تصاویر خودروهای ترافیکی را دارد.

همچنین لایه‌ی fc به طور کامل برای مسئله‌ی جدید آموزش داده می‌شود تا بتواند ویژگی‌های استخراج‌شده را به یکی از 8 کلاس موجود در مجموعه داده نگاشت کند.

فریز کردن لایه‌های شبکه

یکی از بخش‌های مهم این آزمایش، فریز کردن بخش‌های ابتدایی شبکه است.

فریز کردن به این معنی است که پارامترهای آن بخش در طول فرآیند آموزش به‌روزرسانی نمی‌شوند.

در این آزمایش، ابتدا تمام پارامترهای مدل غیرفعال شدند:

requires_grad = False

این کار باعث می‌شود پارامترهای بخش‌های فریزشده در فرآیند آموزش به‌روزرسانی نشوند. سپس پارامترهای layer4 مجدداً فعال شدند:

```
layer4.requires_grad = True
```

و همین کار برای لایه‌ی طبقه‌بندی نهایی نیز انجام شد:

```
fc.requires_grad = True
```

بنابراین در پایان، تنها پارامترهای layer4 و fc در فرآیند یادگیری قرار داشتند.

این راهبرد باعث می‌شود مدل مجبور نباشد تمام ویژگی‌هایی را که از قبل در شبکه یاد گرفته است، دوباره از ابتدا یاد بگیرد.

به بیان ساده، می‌توان فرآیند را به صورت زیر در نظر گرفت:

ویژگی‌های عمومی

→

فریز

→

حفظ می‌شوند

در حالی که:

ویژگی‌های عمیق‌تر

→

آموزش

→

با مسئله‌ی جدید سازگار می‌شوند

و در نهایت:

ویژگی‌ها

→

FC



8 کلاس

استفاده از نرخ یادگیری متفاوت

یکی از نکات مهم در این آزمایش، استفاده از دو نرخ یادگیری متفاوت برای layer4 و fc است.

در کد، نرخ یادگیری برای layer4 برابر با 0.0001 در نظر گرفته شد.

نرخ یادگیری برای fc نیز برابر با 0.001 در نظر گرفته شد.

بنابراین نرخ یادگیری لایه‌ی fc برابر با 10 برابر نرخ یادگیری layer4 است.

این انتخاب با این هدف انجام شده است که لایه‌ی طبقه‌بندی که برای مسئله‌ی جدید ایجاد شده است، بتواند با سرعت بیشتری خود را با 8 کلاس موجود در مجموعه داده سازگار کند.

در مقابل، layer4 بخشی از شبکه‌ی ازپیش‌آمورخته شده است و به تغییرات کنترل‌شده‌تری نیاز دارد.

بنابراین می‌توان این راهبرد را به صورت زیر خلاصه کرد:

layer4: تغییرات کوچک‌تر

fc: سازگاری سریع‌تر با مسئله‌ی جدید

خلاصه‌ی معماری مورد استفاده

ساختار آموزش ResNet18 در این آزمایش را می‌توان به صورت زیر خلاصه کرد:

بخش شبکه	وضعیت	نرخ یادگیری
conv1	فریز	—
bn1	فریز	—
layer1	فریز	—
layer2	فریز	—
layer3	فریز	—
layer4	قابل آموزش	0.0001
fc	قابل آموزش	0.001

بنابراین تنها بخش کوچکی از کل شبکه در طول آموزش تغییر می‌کند و بخش عمده‌ی ویژگی‌های ازپیش‌آمورخته شده حفظ می‌شود.

این روش یکی از شکل‌های رایج استفاده از Transfer Learning است که در آن به جای آموزش کامل یک شبکه‌ی عمیق، تنها بخش‌هایی از شبکه که برای مسئله‌ی جدید اهمیت بیشتری دارند، تنظیم مجدد می‌شوند.

نتایج آزمایش ResNet18

پس از اعمال راهبرد Transfer Learning و آموزش بخش‌های layer4 و fc، آزمایش با ترکیب‌های مختلف افزایش داده Augmentation انجام شد.

در میان آزمایش‌های انجام‌شده، بهترین عملکرد مربوط به مدلی بود که از روش افزایش داده‌ی Full without Color استفاده می‌کرد.

جزئیات بهترین اجرای به‌دست‌آمده به صورت زیر است:

مدل: ResNet18 - Layer4 Fine-Tuning

نوع افزایش داده: Full without Color

Seed: 42

Accuracy: Validation 97.08%

فایل مدل ذخیره‌شده: resnet18_layer4_best.pth

بر اساس نتایج به‌دست‌آمده، بهترین دقت اعتبارسنجی این آزمایش برابر با 97.08% بوده است.

این نتیجه نشان می‌دهد که استفاده از Transfer Learning و آموزش مجدد بخش layer4 به همراه لایه‌ی طبقه‌بندی نهایی می‌تواند عملکرد بسیار مناسبی برای مسئله‌ی طبقه‌بندی تصاویر خودروهای ترافیکی ایجاد کند.

در این آزمایش، بخش‌های ابتدایی ResNet18 همچنان فریز باقی ماندند و تنها بخش‌های عمیق‌تر شبکه که قابلیت سازگاری بیشتری با مسئله‌ی جدید دارند، به همراه لایه‌ی fc آموزش داده شدند.

همچنین استفاده از Full without Color به عنوان بهترین روش افزایش داده نشان می‌دهد که در این آزمایش، ترکیبی از افزایش داده‌های مختلف، بدون اعمال تغییرات رنگی، توانسته است بهترین عملکرد را روی مجموعه‌ی اعتبارسنجی ایجاد کند.

مدل به‌دست‌آمده با نام زیر ذخیره شد:

resnet18_layer4_best.pth

و به عنوان بهترین مدل این بخش از آزمایش مورد استفاده قرار گرفت.

پارامتر	بهترین نتیجه
مدل	ResNet18 - Layer4
Augmentation	Full without Color
Seed	42
Accuracy Validation	97.08%
File Model	resnet18_layer4_best.pth

بنابراین، در ادامه‌ی پروژه می‌توان مدل ResNet18 با تنظیمات فوق را به عنوان مدل منتخب این مرحله در نظر گرفت و عملکرد آن را با شبکه‌ی CustomCNN و سایر آزمایش‌های انجام‌شده مقایسه کرد.

آموزش کامل ResNet18 و Full Fine-Tuning

در آزمایش بعدی، به جای فریز کردن بخش‌های ابتدایی شبکه، تمام پارامترهای معماری ResNet18 برای آموزش مجدداً فعال شدند.

در این روش، برخلاف حالت قبل که تنها layer4 و fc قابل آموزش بودند، تمام لایه‌های شبکه در فرآیند Backpropagation شرکت می‌کنند و وزن‌های آن‌ها متناسب با داده‌های مسئله‌ی جدید به‌روزرسانی می‌شوند.

به این روش Full Fine-Tuning گفته می‌شود.

ساختار آموزش مدل در این حالت به صورت زیر است:

conv1, bn1, layer1, layer2, layer3, layer4, fc

→

قابل آموزش

بنابراین برخلاف روش قبلی، هیچ‌یک از بخش‌های اصلی شبکه در این آزمایش فریز نشده‌اند.

هدف از این آزمایش بررسی این موضوع بود که آیا امکان سازگار شدن کامل ویژگی‌های یادگرفته‌شده‌ی ResNet18 با مجموعه‌داده‌ی خودروهای ترافیکی می‌تواند عملکرد مدل را نسبت به حالت آموزش محدودتر layer4 + fc بهبود دهد یا خیر.

تنظیمات آزمایش Full Fine-Tuning

برای بررسی عملکرد Full Fine-Tuning، تمام ترکیب‌های افزایش داده‌ی تعریف‌شده در آزمایش‌های قبلی مورد استفاده قرار گرفتند.

همچنین برای بررسی تأثیر مقداردهی اولیه و تصادفی بودن فرآیند آموزش، آزمایش‌ها با سه مقدار مختلف Seed انجام شدند.

مقادیر مورد استفاده برای Seed عبارت بودند از:

20 10 42

در هر اجرا، مدل ResNet18 با 8 خروجی متناسب با تعداد کلاس‌های مسئله ایجاد شد.

تنظیمات اصلی آموزش به صورت زیر بود:

پارامتر	مقدار
مدل	ResNet18 Full
روش آموزش	Full Fine-Tuning
تعداد کلاس‌ها	8
Size Batch	32
Optimizer	Adam
Rate Learning	0.0001
Epochs Maximum	500
Patience Stopping Early	75
Seeds	42, 10, 20

در هر ترکیب از Augmentation و Seed، مدل از ابتدا ایجاد و فرآیند آموزش انجام شد. سپس بهترین Validation Accuracy ثبت و مدل متناظر با بهترین عملکرد ذخیره شد.

بهترین نتیجه‌ی ResNet18 Full

پس از اجرای آزمایش‌ها با ترکیب‌های مختلف افزایش داده و سه مقدار مختلف Seed، بهترین عملکرد مربوط به استفاده از روش Crop با Seed = 42 بود.

جزئیات بهترین اجرای به‌دست‌آمده به صورت زیر است:

مدل: ResNet18 Full Fine-Tuning

نوع افزایش داده: Crop

Seed: 42

Accuracy: Validation 98.20%

فایل مدل ذخیره‌شده: resnet18_full_best.pth

بهترین دقت اعتبارسنجی به‌دست‌آمده در این آزمایش برابر با:

$$\text{Validation Accuracy} = 98.20\%$$

بود.

این نتیجه نشان می‌دهد که آموزش کامل شبکه‌ی ResNet18 توانسته است عملکرد بسیار بالایی روی مجموعه‌ی اعتبارسنجی ایجاد کند.

در این آزمایش، برخلاف روش Layer4 Fine-Tuning، تمام لایه‌های شبکه امکان به‌روزرسانی داشتند و بنابراین شبکه می‌توانست علاوه بر لایه‌های انتهایی، ویژگی‌های استخراج‌شده در لایه‌های ابتدایی‌تر را نیز با ویژگی‌های خاص مجموعه‌داده‌ی خودروهای ترافیکی سازگار کند.

تحلیل بهترین روش افزایش داده

در میان روش‌های مختلف افزایش داده، حالت Crop بهترین عملکرد را در آزمایش ResNet18 Full ایجاد کرد. در این روش از Random Resized Crop استفاده می‌شود. اندازه‌ی خروجی این تبدیل برابر با:

$$150 \times 150$$

است.

در این روش، بخشی تصادفی از تصویر با مقیاس بین 70% تا 100% انتخاب شده و سپس به اندازه‌ی 150x150 تغییر داده می‌شود.

نتیجه‌ی به‌دست‌آمده نشان می‌دهد که این نوع تغییر در مقیاس و ناحیه‌ی تصویر در آزمایش حاضر برای یادگیری ویژگی‌های مرتبط با کلاس‌های خودرو مناسب بوده است.

با این حال، بهتر است این نتیجه به مجموعه داده و تنظیمات فعلی محدود شود و نمی‌توان صرفاً بر اساس این آزمایش نتیجه گرفت که Crop در تمام مسائل طبقه‌بندی خودرو بهترین روش افزایش داده است.

مقایسه با Layer4 Fine-Tuning

در آزمایش قبلی، تنها layer4 و fc قابل آموزش بودند و بهترین دقت اعتبارسنجی برابر با 97.08% به دست آمد.

در آزمایش حاضر، با استفاده از Full Fine-Tuning و قابل آموزش بودن تمام پارامترهای شبکه، بهترین دقت اعتبارسنجی به 98.20% رسید.

بنابراین اختلاف عملکرد دو روش برابر است با:

$$1.12\% = 97.08\% - 98.20\%$$

در نتیجه، Full Fine-Tuning در این آزمایش توانست حدود 1.12 واحد درصد دقت اعتبارسنجی بالاتری نسبت به Layer4 ایجاد کند.

روش	Augmentation	Seed	Accuracy Validation
Layer4 Fine-Tuning	Full without Color	42	97.08%
Full Fine-Tuning	Crop	42	98.20%

این مقایسه نشان می‌دهد که در مجموعه داده‌ی مورد استفاده در این پروژه، اجازه دادن به تمام لایه‌های ResNet18 برای سازگار شدن با داده‌های جدید، عملکرد اعتبارسنجی بهتری نسبت به آموزش محدود layer4 و fc ایجاد کرده است.

بنابراین در ادامه‌ی پروژه، مدل ResNet18 با روش Full Fine-Tuning و تنظیمات مربوط به بهترین اجرای آن به عنوان مدل منتخب این مرحله در نظر گرفته شد.

مدل‌های مورد استفاده در Ensemble روی دیتاست دوم

لازم به ذکر است که تمام نتایج ارائه‌شده در این بخش مربوط به **دیتاست دوم** پروژه هستند. این دیتاست با مسیر CROP+MERGED/train مورد استفاده قرار گرفته و تمامی مدل‌های زیر بر اساس همین نسخه از داده‌ها آموزش یا ارزیابی شده‌اند.

در مرحله‌ی نهایی، پنج فایل مدل برای تشکیل Weighted Ensemble مورد استفاده قرار گرفتند. این پنج فایل لزوماً نشان‌دهنده‌ی پنج معماری کاملاً متفاوت نیستند، بلکه شامل چند اجرای مختلف از مدل ResNet18 Full Fine-Tuning و همچنین یک مدل با ورودی نرمال‌شده و یک مدل Layer4 Fine-Tuning هستند.

مدل‌های مورد استفاده عبارت‌اند از:

Accuracy Validation	Type Training	Model
98.09%	Full Fine-Tuning	resnet18_d2_full_1.pth
98.43%	Full Fine-Tuning	resnet18_d2_full_2.pth
98.20%	Full Fine-Tuning	resnet18_d2_full_best.pth
97.08%	Layer4 Fine-Tuning	resnet18_d2_layer4_best.pth
97.76%	Full Fine-Tuning + Normalization	resnet18_d2_fullnormalized.pth

تکرار آموزش Full Fine-Tuning با تصادفی‌بودن متفاوت

دو مدل زیر:

resnet18_d2_full_1.pth

9

resnet18_d2_full_2.pth

در واقع مربوط به دو اجرای مستقل از یک آزمایش یکسان هستند.

در هر دو اجرا، معماری و تنظیمات اصلی مدل یکسان بوده و از روش Full Fine-Tuning روی دیتاست دوم استفاده شده است. تفاوت اصلی میان این دو اجرا در وجود فرآیندهای تصادفی آموزش، از جمله مقداردهی اولیه و سایر عوامل تصادفی فرآیند آموزش است.

بنابراین می‌توان این دو مدل را به صورت زیر در نظر گرفت:

Full Fine-Tuning + Random Run 1

9

Full Fine-Tuning + Random Run 2

هدف از اجرای مجدد یک آزمایش یکسان این است که مشخص شود عملکرد مدل تا چه اندازه به تصادفی بودن فرآیند آموزش وابسته است.

نتایج نشان می‌دهند که حتی با استفاده از یک معماری و تنظیمات یکسان، دو اجرای مستقل می‌توانند دقت‌های متفاوتی ایجاد کنند:

Accuracy Validation	Run
98.09%	Full Run 1
98.43%	Full Run 2

اختلاف این دو اجرا برابر است با:

$$0.34\%$$

زیرا:

$$0.34\% = 98.09\% - 98.43\%$$

این اختلاف نشان می‌دهد که بخشی از تغییرات عملکرد می‌تواند ناشی از تصادفی بودن فرآیند آموزش باشد. بنابراین استفاده از یک اجرای منفرد همیشه تصویر کاملی از رفتار مدل ارائه نمی‌دهد.

مدل Full Best

فایل:

resnet18_d2_full_best.pth

نیز مربوط به مدل ResNet18 Full Fine-Tuning روی دیتاست دوم است.

این مدل به عنوان بهترین مدل ذخیره شده از اجرای مربوط به Full Fine-Tuning در نظر گرفته شده و دقت اعتبارسنجی آن برابر با:

$$98.20\%$$

است.

بنابراین این مدل از نظر معماری با دو اجرای full_1 و full_2 تفاوت اساسی ندارد، بلکه یک checkpoint منتخب از آزمایش‌های Full Fine-Tuning است.

مدل Layer4 Fine-Tuning

فایل:

resnet18_d2_layer4_best.pth

مربوط به آزمایش قبلی Transfer Learning است که در آن بخش‌های ابتدایی شبکه فریز شده و تنها بخش layer4 به همراه لایه‌ی fc آموزش داده شده‌اند. دقت اعتبارسنجی این مدل برابر با:

97.08%

است.

بنابراین این مدل از نظر روش آموزش با مدل‌های Full Fine-Tuning متفاوت است و نماینده‌ی روش آموزش محدودتر Layer4 Fine-Tuning در Ensemble محسوب می‌شود.

مدل Normalization با Full Fine-Tuning

فایل:

resnet18_d2_fullnormalized.pth

نیز مربوط به Full Fine-Tuning روی دیتاست دوم است، اما این بار تصاویر ورودی با مقادیر استاندارد ImageNet نرمال‌سازی شده‌اند. در این حالت، علاوه بر تغییر اندازه‌ی تصاویر به:

150×150

عملیات زیر نیز روی تصاویر انجام شده است:

Normalize ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$)

بنابراین تفاوت این مدل با سایر مدل‌های Full در نحوه‌ی آماده‌سازی ورودی شبکه است. دقت اعتبارسنجی این مدل برابر با:

97.76%

به دست آمده است.

به عبارت دیگر، این آزمایش بررسی می‌کند که آیا استفاده از نرمال‌سازی استاندارد تصاویر می‌تواند عملکرد Full Fine-Tuning را در دیتاست دوم بهبود دهد یا خیر.

جمع‌بندی مدل‌های Ensemble

در نتیجه، پنج مدل استفاده‌شده در Ensemble را می‌توان به صورت

زیر دسته‌بندی کرد:

Accuracy Validation	Type Experiment	Model
98.09%	Full + Random Run 1	full_1
98.43%	Full + Random Run 2	full_2
98.20%	Best Checkpoint from Full	full_best
97.08%	Layer4 Fine-Tuning	layer4_best
97.76%	Full + Normalization	fullnormalized

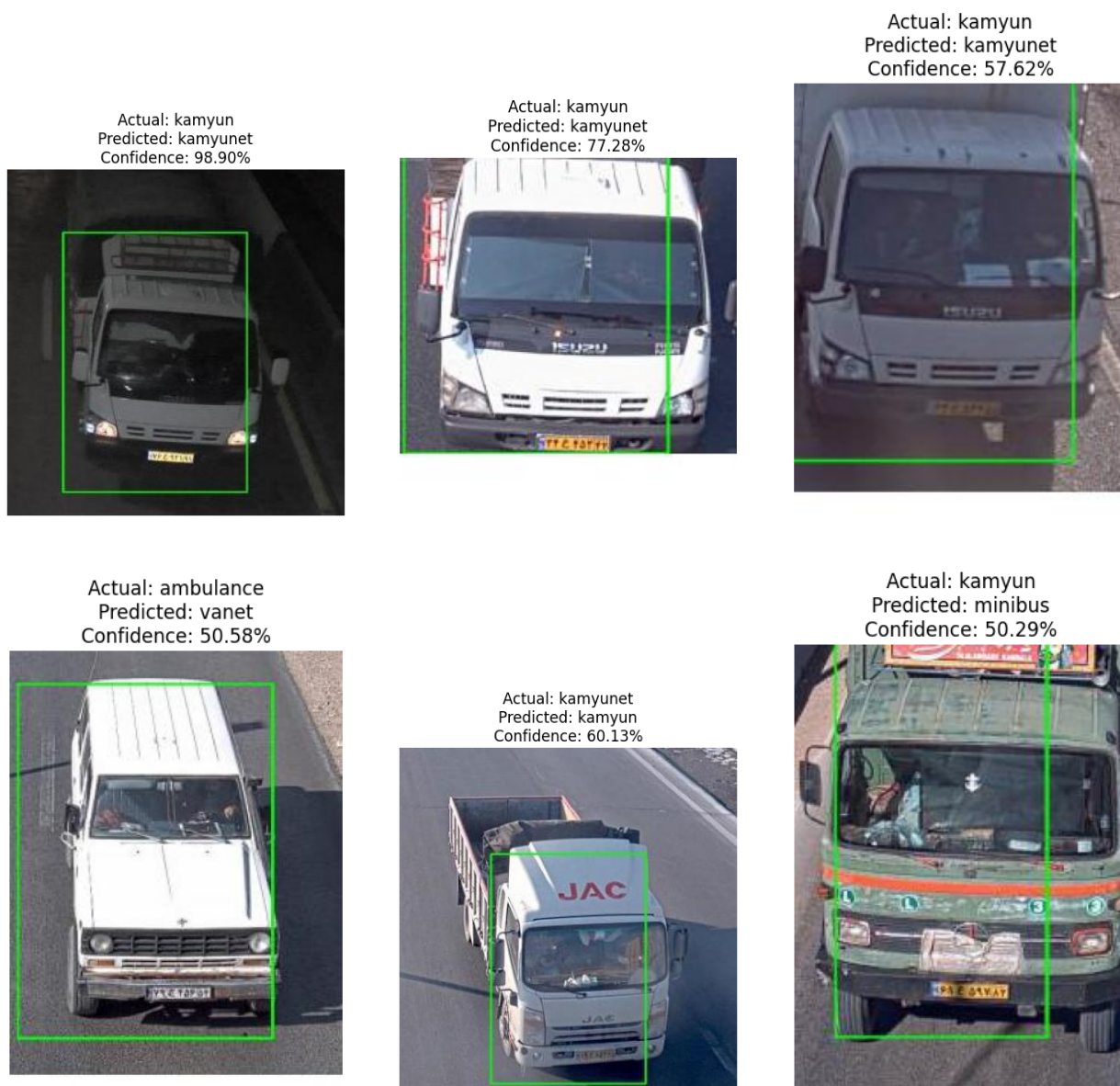
بنابراین، Ensemble نهایی ترکیبی از چند اجرای مدل ResNet18 روی **دیتاست دوم** است که تفاوت آن‌ها در روش Fine-Tuning، اجرای تصادفی و نحوه‌ی آماده‌سازی تصاویر ورودی قرار دارد.

در نهایت، خروجی احتمالی این پنج مدل با استفاده از وزن‌هایی که بر اساس Validation Accuracy آن‌ها محاسبه شدند، ترکیب شد. نتیجه‌ی این ترکیب روی مجموعه‌ی تست شامل ۴۰۰ تصویر برابر با:

$$\frac{394}{400} = 98.5\%$$

به دست آمد.

در نتیجه، سیستم Ensemble توانست ۳۹۴ تصویر را به درستی طبقه‌بندی کند و تنها ۶ تصویر را اشتباه تشخیص دهد. در ادامه، شش نمونه از تصاویری که توسط مدل Ensemble به اشتباه طبقه‌بندی شده‌اند، نمایش داده شده است. این تصاویر نمونه‌هایی از خطاهای مدل در مجموعه‌ی تست هستند و می‌توانند برای بررسی نقاط ضعف مدل و تحلیل عواملی که باعث تشخیص نادرست شده‌اند مورد استفاده قرار گیرند.



شکل ۳: نمونه‌هایی از تصاویر اشتباه طبقه‌بندی‌شده توسط مدل Ensemble

جمع‌بندی نهایی

در این پروژه، عملکرد مدل در چندین آزمایش مختلف بررسی شد و تأثیر روش‌های گوناگون آموزش و تنظیم مدل مورد ارزیابی قرار گرفت. نتایج حاصل از این آزمایش‌ها در جدول زیر خلاصه شده‌اند.

Class Lowest-Precision	Class Lowest-Recall	F1 Macro	Recall Macro	Precision Macro	Experiment
kamyunet	kamyun	93.32%	93.26%	93.45%	CNN baseline
kamyun	minibus	91.89%	91.69%	92.32%	Balanced batches
kamyunet	kamyun	92.00%	91.96%	92.17%	BCE
kamyun	kamyunet	87.80%	87.49%	88.28%	Cross-Entropy
kamyunet	kamyun	92.92%	92.88%	93.09%	Scheduler
kamyun	kamyun	93.82%	93.75%	93.92%	Weight Decay

با بررسی نتایج جدول، می‌توان مشاهده کرد که تغییر تنظیمات و روش‌های آموزشی مختلف تأثیر قابل توجهی بر عملکرد شبکه‌ی عصبی سفارشی داشته است. مدل پایه‌ی CNN با مقدار Macro Precision برابر با 93.45%، مقدار Macro Recall برابر با 93.26% و مقدار Macro F1 برابر با 93.32% عملکرد مناسبی را به عنوان نقطه‌ی شروع آزمایش‌ها نشان داده است.

در این آزمایش، فاصله‌ی بسیار کمی بین سه معیار اصلی مشاهده می‌شود. این موضوع نشان می‌دهد که عملکرد مدل پایه از نظر میزان صحت پیش‌بینی‌ها و توانایی شناسایی نمونه‌های متعلق به کلاس‌های مختلف نسبتاً متعادل بوده است. مقدار Macro F1 برابر با 93.32% نیز نشان می‌دهد که عملکرد مدل تنها به چند کلاس خاص وابسته نبوده و در میان کلاس‌ها عملکرد کلی مناسبی داشته است.

تحلیل تأثیر Weight Decay

بهترین نتیجه در میان آزمایش‌های انجام‌شده روی شبکه‌ی عصبی سفارشی مربوط به استفاده از روش Weight Decay است.

در این آزمایش مقدار Macro Precision به 93.92%، مقدار Macro Recall به 93.75% و مقدار Macro F1 به 93.82% رسید.

در مقایسه با مدل پایه، مقدار Macro F1 از 93.32% به 93.82% افزایش پیدا کرده است. بنابراین بهبود حاصل‌شده برابر با 0.50 واحد درصد است.

همچنین مقدار Macro Precision نسبت به مدل پایه 0.47 واحد درصد و مقدار Macro Recall نیز 0.49 واحد درصد افزایش پیدا کرده است.

این نتیجه نشان می‌دهد که در مجموعه‌داده‌ی مورد استفاده، اعمال منظم‌سازی به وسیله‌ی Weight Decay توانسته است عملکرد کلی شبکه را اندکی بهبود دهد.

یکی از دلایل احتمالی این بهبود، جلوگیری از وابستگی بیش از حد مدل به ویژگی‌های خاص نمونه‌های آموزشی است. در نتیجه، مدل می‌تواند ویژگی‌های عمومی‌تری را یاد بگیرد و در هنگام ارزیابی روی داده‌های جدید عملکرد پایدارتری داشته باشد.

تحلیل Balanced Batches

در آزمایش مربوط به Balanced Batches، مقدار Macro F1 برابر با 91.89% به دست آمد که نسبت به مدل پایه با مقدار 93.32%، برابر با 1.43 واحد درصد کاهش دارد.

همچنین Macro Precision از 93.45% به 92.32% و Macro Recall از 93.26% به 91.69% کاهش یافته است. بنابراین در این مجموعه داده، متعادل کردن نحوه‌ی تشکیل دسته‌های آموزشی نتوانسته است باعث بهبود عملکرد کلی مدل شود.

این نتیجه نشان می‌دهد که صرفاً ایجاد توازن بیشتر در دسته‌های آموزشی الزاماً به معنای افزایش عملکرد مدل نیست و تأثیر آن به نحوه‌ی توزیع داده‌ها، تعداد نمونه‌های هر کلاس و ویژگی‌های خود مجموعه داده نیز وابسته است.

تحلیل BCE

در آزمایش مربوط به تابع خطای BCE، مقدار Macro F1 برابر با 92.00% به دست آمد.

این مقدار نسبت به مدل پایه 1.32 واحد درصد کمتر است.

همچنین مقدار Macro Precision برابر با 92.17% و مقدار Macro Recall برابر با 91.96% بوده است.

بنابراین تغییر تابع خطا از تنظیم پایه به BCE در این آزمایش نتوانسته است عملکرد شبکه را بهبود دهد.

این نتیجه را می‌توان به این صورت تفسیر کرد که انتخاب تابع خطا باید متناسب با نوع مسئله و ساختار خروجی مدل انجام شود و تغییر تابع خطا به تنهایی تضمین‌کننده‌ی بهبود عملکرد نیست.

تحلیل Cross-Entropy

ضعیف‌ترین نتیجه در میان آزمایش‌های جدول مربوط به آزمایش Cross-Entropy است.

در این حالت مقدار Macro Precision برابر با 88.28%، مقدار Macro Recall برابر با 87.49% و مقدار Macro F1 برابر با 87.80% به دست آمده است.

در مقایسه با مدل پایه، مقدار Macro F1 حدود 5.52 واحد درصد کاهش داشته است.

همچنین Macro Recall به 87.49% کاهش یافته که کمترین مقدار Recall در میان تمام آزمایش‌های جدول است.

بنابراین می‌توان نتیجه گرفت که تنظیمات استفاده‌شده در این آزمایش با Cross-Entropy برای شبکه‌ی سفارشی مورد استفاده در این پروژه مناسب نبوده است.

با توجه به اختلاف قابل توجه این نتیجه با سایر آزمایش‌ها، مشخص است که انتخاب تنظیمات آموزشی و تابع خطا می‌تواند تأثیر بسیار بیشتری از یک تغییر جزئی در معماری شبکه بر عملکرد نهایی داشته باشد.

تحلیل Learning Rate Scheduler

آزمایش Scheduler با مقدار Macro F1 برابر با 92.92% عملکردی نزدیک به مدل پایه داشته است.

در این آزمایش مقدار Macro Precision برابر با 93.09% و مقدار Macro Recall برابر با 92.88% بوده است.

مقایسه‌ی این نتایج با مدل پایه نشان می‌دهد که استفاده از Learning Rate Scheduler در تنظیمات فعلی، بهبود قابل

توجهی در معیار Macro F1 ایجاد نکرده است.

در واقع مقدار Macro F1 نسبت به مدل پایه حدود 0.40 واحد درصد کمتر شده است.

بنابراین اگرچه تغییر نرخ یادگیری می‌تواند در برخی شرایط باعث همگرایی بهتر مدل شود، اما در این آزمایش خاص استفاده از Scheduler به تنهایی باعث بهبود عملکرد نهایی شبکه نشده است.

مقایسه‌ی کلی آزمایش‌ها

اگر معیار Macro F1 را به عنوان معیار اصلی مقایسه در نظر بگیریم، ترتیب عملکرد آزمایش‌ها به صورت زیر است:

Cross-Entropy < Balanced batches < BCE < Scheduler < CNN baseline < Weight Decay

بهترین مقدار Macro F1 برابر با 93.82% و مربوط به Weight Decay است، در حالی که ضعیف‌ترین مقدار برابر با 87.80% و مربوط به Cross-Entropy است.

بنابراین فاصله‌ی بین بهترین و ضعیف‌ترین آزمایش برابر با 6.02 واحد درصد است.

این اختلاف نشان می‌دهد که انتخاب تنظیمات مناسب آموزش می‌تواند تأثیر قابل توجهی بر عملکرد یک شبکه‌ی عصبی داشته باشد.

تحلیل عملکرد کلاس‌ها

علاوه بر معیارهای کلی، بررسی کلاس‌هایی که کمترین مقدار Precision و Recall را دارند نیز اطلاعات مهمی درباره‌ی رفتار مدل ارائه می‌دهد.

در بیشتر آزمایش‌ها، کلاس kamyun به عنوان کلاسی با کمترین Recall ظاهر شده است.

این موضوع نشان می‌دهد که مدل در شناسایی صحیح نمونه‌های واقعی کلاس kamyun با دشواری بیشتری مواجه بوده است. به عبارت دیگر، بخشی از تصاویر واقعی این کلاس توسط مدل به عنوان کلاس‌های دیگر پیش‌بینی شده‌اند.

در آزمایش Balanced Batches، کمترین Recall مربوط به کلاس minibus بوده است و در آزمایش Cross-Entropy نیز کلاس kamyunet کمترین مقدار Recall را داشته است.

بنابراین تغییر تنظیمات آموزشی می‌تواند علاوه بر تغییر عملکرد کلی مدل، بر میزان موفقیت مدل در شناسایی کلاس‌های خاص نیز تأثیر بگذارد.

از طرف دیگر، کلاس kamyunet در چندین آزمایش به عنوان کلاسی با کمترین Precision ظاهر شده است.

پایین‌تر بودن Precision این کلاس می‌تواند نشان‌دهنده‌ی این باشد که مدل در برخی موارد تصاویر متعلق به کلاس‌های دیگر را نیز به اشتباه به عنوان kamyunet پیش‌بینی کرده است.

در آزمایش Weight Decay، کلاس kamyun همچنان به عنوان یکی از کلاس‌های چالش‌برانگیز مشاهده می‌شود، اما عملکرد کلی مدل در این آزمایش بهترین مقدار را در میان آزمایش‌های شبکه‌ی سفارشی داشته است.

نتیجه‌گیری آزمایش‌های شبکه‌ی سفارشی

در مجموع، نتایج نشان می‌دهند که مدل پایه‌ی CNN از عملکرد اولیه‌ی مناسبی برخوردار بوده است، اما اعمال Weight Decay توانسته است بهترین نتیجه را در میان تنظیمات بررسی‌شده ایجاد کند.

در مقابل، روش‌هایی مانند BCE, Balanced Batches و Scheduler در تنظیمات مورد استفاده نتوانستند عملکرد مدل پایه را بهبود دهند و آزمایش Cross-Entropy نیز کاهش قابل توجهی در عملکرد ایجاد کرد. بنابراین مدل CNN + Weight Decay به عنوان بهترین مدل در میان آزمایش‌های شبکه‌ی عصبی سفارشی انتخاب شد. با این حال، عملکرد این مدل همچنان با عملکرد نهایی روش Ensemble فاصله دارد. به همین دلیل در مرحله‌ی نهایی پروژه از چند مدل ResNet18 در قالب روش Ensemble استفاده شد.

نتیجه‌ی نهایی سیستم

در مرحله‌ی نهایی پروژه، برای افزایش پایداری پیش‌بینی و استفاده از توانایی چند مدل، چند مدل ResNet18 با یکدیگر ترکیب شدند.

استفاده از روش Ensemble باعث شد تصمیم نهایی سیستم تنها بر اساس پیش‌بینی یک مدل انجام نشود و اطلاعات چند مدل در فرآیند تصمیم‌گیری مورد استفاده قرار گیرد.

در مجموعه‌ی تست شامل 400 تصویر، تعداد 394 تصویر به درستی طبقه‌بندی شد.

بنابراین تعداد تصاویر اشتباه برابر با 6 تصویر بوده است.

دقت نهایی سیستم برابر است با:

$$394 / 400 = 98.50\%$$

در نتیجه، دقت نهایی سیستم برابر با 98.50% به دست آمده است.

این مقدار نشان می‌دهد که روش Ensemble در مقایسه با بهترین Macro F1 به دست آمده از شبکه‌ی سفارشی، عملکرد نهایی بسیار بالاتری ایجاد کرده است.

البته باید توجه داشت که 93.82% مربوط به Macro F1 مدل Weight Decay است، در حالی که 98.50% مربوط به دقت نهایی روی مجموعه‌ی تست است. بنابراین این دو مقدار معیارهای یکسانی نیستند و مقایسه‌ی مستقیم آن‌ها به عنوان یک معیار واحد صحیح نیست.

با این وجود، نتایج نهایی نشان می‌دهند که استفاده از معماری از پیش‌آموزته شده‌ی ResNet18، یادگیری انتقالی، تنظیم مناسب فرآیند آموزش و در نهایت ترکیب چند مدل، توانسته است یک سیستم طبقه‌بندی با عملکرد بسیار بالا ایجاد کند.

در مجموع، می‌توان گفت که آزمایش‌های انجام شده روی شبکه‌ی سفارشی برای بررسی اثر تنظیمات مختلف آموزش مفید بودند و بهترین تنظیم این بخش با استفاده از Weight Decay به دست آمد. سپس استفاده از ResNet18 و روش Ensemble امکان دستیابی به عملکرد نهایی بالاتر را فراهم کرد.

پایان گزارش

با تشکر از توجه شما